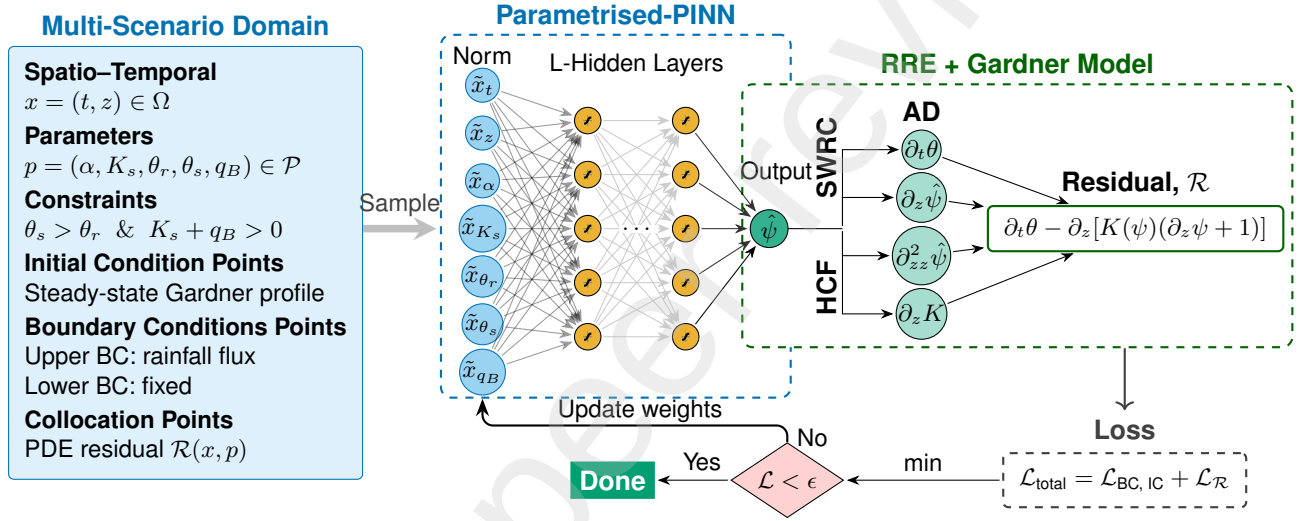


# Graphical Abstract

## Parametrised PINNs for Water Infiltration in Unsaturated Soils

Mohamed Gowely, Anil Yildiz



## Highlights

### **Parametrised PINNs for Water Infiltration in Unsaturated Soils**

Mohamed Gowely, Anil Yildiz

- PINNs have been successfully applied to solve water infiltration in unsaturated soils, but with limited domains in limited parameter spaces.
- This study presents a parametrised PINN application, covering various soil hydraulic parameters and a large space-time domain.
- Although initial computational cost is higher, it pays off when multiple scenarios are of interest.

# Parametrised PINNs for Water Infiltration in Unsaturated Soils

Mohamed Gowely<sup>a</sup>, Anil Yildiz<sup>a,\*</sup>

*<sup>a</sup>Methods for Model-based Development in Computational Engineering, RWTH Aachen University, Aachen, Germany*

---

## Abstract

Solving water infiltration in unsaturated soils has traditionally relied on two approaches: analytical solutions and classical numerical methods. However, the former relies on simplified assumptions which are not the case in realistic field applications, while the latter is limited by spatial and temporal discretisation which remains a major obstacle, due to mass conservation issues and convergence failures. Physics-Informed Neural Networks (PINNs) have recently gained popularity in approximating non-linear functions, as a powerful mesh-free approach, by constraining the training with physical laws and with using sparse data to mimic field sensors which generally require a certain gap to mitigate interference between sensors. Despite several successful applications on modelling this problem, larger spatio-temporal domains, root water uptake, and lack of universality remain unaddressed. This study tests the efficacy of PINNs on larger domains in an unsupervised manner, and incorporates a sink term representing uniform root architecture. Furthermore, it transforms PINNs from a fixed scenario approximator to a

---

\*Corresponding author

Email address: yildiz@mbd.rwth-aachen.de (Anil Yildiz)

universal approximator. This is carried out by parametrising PINNs, where the input space becomes a continuous multi-dimensional space comprising five hydraulic parameters, alongside spatio-temporal coordinates. Training is conducted with the aid of dynamic adaptive functions, yielding an efficient surrogate model. The model results were validated against analytical solution, showing high accuracy with a Nash Sutcliffe Efficiency exceeding 0.99 in 19 out of 21 tested scenarios. The worst scenario scored  $NSE > 0.96$ , corresponding to a minor underestimation of  $\sim 1.0\%$  on the Percent Bias (PBIAS) scale. The training took about  $2.3\times$  longer than standard PINN training, and P-PINN model evaluates new scenarios almost instantly without either retraining, or preprocessing and meshing. Despite this, there were limitations shown as phase errors, which are related to spectral bias and temporal causality. Nevertheless, this study shows the practicality of P-PINNs, as an efficient universal approximator for modelling infiltration problems.

*Keywords:* Physics-informed neural networks; Unsaturated flow; Richardson-Richards equation; Parametric modelling; Deep learning; Root water uptake

---

## 1. Introduction

Soil infiltration governs the balance between surface run-off, evaporation, groundwater recharge (Zhang et al., 2018), and directly relates to managing the risk of predominantly shallow landslides and flood occurrence (Iverson, 2000; Chahinian et al., 2005). Moreover, a precise estimation of soil moisture levels in soil requires prediction of soil infiltration, which can then be used to efficiently regulate the water needed for optimum plant growth. There-

fore, it is vital for keeping a healthy ecosystem and sustainable agriculture (Sharma et al., 2025). Predicting the outcomes of the infiltration process in soil hydrology can be efficiently conducted by using the Richardson-Richards Equation (RRE). However, solving the RRE at a large scale is computationally expensive (Farthing and Ogden, 2017), and coupling with root water uptake further increases complexity (Vanderborght et al., 2024; Yuan et al., 2025). With recent Machine Learning (ML) and Graphics Processing Units (GPU) advancements, Physics-Informed Neural Networks (PINNs) have been used as an alternative mesh-free approach by approximating solutions to non-linear problems by constraining the model with physical laws and learning from sparse data (Raissi et al., 2019; Karniadakis et al., 2021).

Existing research on using PINNs for the 1D RRE is split between forward and inverse modelling. Inverse models require data points distributed throughout the spatio-temporal domain, with collocation-to-data points ratio ( $N_f : N_{data}$ ) ranging from 1 : 1 (Elkhadrawi et al., 2024; Bandai and Ghezzehei, 2021) to 13 : 1 (38,076 : 3,006) (Depina et al., 2021). While data supplied in the forward models are mainly for the initial and boundary conditions with ratios varying from 4 : 1 (2,000 : 500) (Li et al., 2025) to 8 : 1 (10,000 : 1,200) (Bandai and Ghezzehei, 2022b). No data usage was also reported in Chen et al. (2023).

Aforementioned studies have improved the compliance with physics by imposing monotonicity in the neural network architecture (Bandai and Ghezzehei, 2021), dynamic loss balancing to auto-tune the weights (Chen et al., 2023), double-loop optimisation for filtering the best parameters to arrive at a global minima (Depina et al., 2021), and data regularised training (Li

et al., 2025); yet three main limitations prevail.

Firstly, there is a clear gap in testing the potential of PINNs in solving the RRE over larger scales in one dimension, since existing studies for forward modelling of homogenous soils cases were  $[10 \text{ cm} \times 10 \text{ h}]$  (Chen et al., 2023; Bandai and Ghezzehei, 2022b), and  $[100 \text{ cm} \times 5 \text{ h}]$  (Liu et al., 2024). Secondly, the potential of solving the forward problem with PINNs for plant root uptake has not been tested extensively in the literature. Thirdly, and most importantly, there is a lack of generalisation, since existing PINNs models are trained for a fixed set of soil parameters, and changing any parameter requires re-training the model from scratch. This need of re-training disables the universal approximator property of neural networks, and hinders the widespread applications of PINNs for water infiltration problems.

This study aims to demonstrate the potential and limitations of the PINNs for solving RRE in 1D over large spatio-temporal domains by adopting adaptive functions that have proven successful in the literature. The objectives are (i) to integrate a sink term as a function of depth in the residual of the PDE accounting for root water uptake, and (ii) to parametrise PINNs to train over a continuous range of soil types and hydrodynamic properties. Finally, we evaluate the model's generalisation on different scenarios to evaluate its potential for being universal approximators.

## 2. Mathematical Formulation

Richards (1931) formulated the equation governing unsaturated flow in a vertical soil profile by combining Darcy's law with the continuity equation (Richards, 1931; Zhang et al., 2016), and is expressed as the one-dimensional

Richardson-Richards Equation (1D RRE):

$$\frac{\partial \theta}{\partial t} = \frac{\partial}{\partial z} \left[ K(\psi) \left( \frac{\partial \psi}{\partial z} + 1 \right) \right] - S(z, t). \quad (1)$$

where  $\theta$  is the volumetric water content (dimensionless or in percentage),  $\psi$  is the pressure head (m),  $K(\psi)$  is the unsaturated hydraulic conductivity (generally in  $\text{m s}^{-1}$ ), and  $S(z, t)$  is a sink term representing root water uptake or evaporation ( $\text{s}^{-1}$ ). For non-vegetated soil with negligible surface evaporation,  $S = 0$ .

The RRE is a highly non-linear, elliptic-parabolic partial differential equation (List and Radu, 2016), where it transitions from parabolic to elliptic when the soil becomes saturated (Farthing and Ogden, 2017). The difficulty in numerically solving the RRE stems from the non-linearity of the constitutive relationships relating  $\theta$  and  $K$  with  $\psi$  (Wu et al., 2020).

There are two main constitutive relationships that describe the hydraulic behaviour of unsaturated soil. One relates the state variables to each other,  $\theta = f(\psi)$ , which is generally called the Soil Water Retention Curve (SWRC). The other relates the hydraulic conductivity to a state variable,  $K = g(\psi)$ , and is called the Hydraulic Conductivity Function (HCF). In both relationships, as the soil dries,  $\theta$  decreases, while  $K$  drops several orders of magnitude, hence suction increases (Gardner, 1958), and is expressed mathematically as:

$$K(\psi) = K_s \exp(\alpha\psi), \quad (2a)$$

$$\theta(\psi) = \theta_r + (\theta_s - \theta_r) \exp(\alpha\psi), \quad (2b)$$

where  $K_s$  is the saturated hydraulic conductivity,  $\alpha$  is Gardner's soil pore-size distribution ( $\text{m}^{-1}$ ).  $\theta_s$  and  $\theta_r$  are the saturated and residual volumetric water content, respectively.

The exponential form of Eq. (2) is what linearises the RRE, using the Kirchhoff integral transformation as demonstrated by [Srivastava and Yeh \(1991\)](#). It is known for its simplicity ([Wu et al., 2020](#)), as it has been widely used in developing analytical solutions ([Yuan and Lu, 2005](#); [Ng et al., 2015](#); [Srivastava and Yeh, 1991](#)). There is a limitation, however, and that is, as soon as suction is applied, an exponential decay is immediate ([Wu et al., 2020](#)). Hence, the model fails to capture the suction needed to overcome capillary forces ([Van Genuchten, 1980](#)).

In order to find a unique solution to RRE in one dimension, we defined Initial Condition (IC) and Boundary Conditions (BC) similar to [Yuan and Lu \(2005\)](#):

$$\psi(z, t = 0) = \psi_0(z), \quad (3a)$$

$$\psi(z_{lb}, t) = \psi_{lb}, \quad (3b)$$

$$\left[ K(\psi) \left( \frac{\partial \psi}{\partial z} + 1 \right) \right] z_{ub} = -q(t). \quad (3c)$$

IC follows Gardner's steady-state solution in Eq. (2b). The lower boundary condition (BC), defined in Eq. (3b), is a Dirichlet condition imposing a constant pressure head  $\psi_{lb}$  at the bottom boundary  $z_{lb}$ , representing a water table where  $\psi = 0$ . The upper BC, Eq. (3c), is a Neumann condition for  $\psi$ , where the gradient of  $\psi$  is prescribed by a time-dependent surface flux  $q(t)$  at  $z = z_{ub}$ , representing phenomena such as rainfall ( $q < 0$ ) or evaporation

( $q > 0$ ) (Yuan and Lu, 2005).

To verify the PINNs' predictions, the analytical solutions developed by Yuan and Lu (2005) are adopted herein as a basis for comparison. The RRE Eq. (1) is solved for a vertical domain  $z \in [-L, 0]$  using Gardner's model Eq. (2) for linearisation. The flux potential  $\Phi$  and diffusivity  $D$  are defined as:

$$\Phi(z, t) = \frac{K_s}{\alpha} \exp(\alpha\psi), \quad D = \frac{K_s}{\alpha(\theta_s - \theta_r)}. \quad (4)$$

For a non-vegetated profile, the steady-state potential  $\Phi_s$  under an initial surface flux  $q_0$  and fixed bottom boundary  $\psi_1$  is given by:

$$\Phi_s(z) = \frac{K_s}{\alpha} \exp[\alpha(\psi_1 - z)] + \frac{q_0}{\alpha} [\exp(-\alpha z) - 1]. \quad (5)$$

When the surface flux switches to  $q_1$  at  $t > 0$ , the transient potential is obtained via eigenfunction expansion:

$$\Phi(z, t) = \Phi_{s, q_1}(z) - 8(q_0 - q_1) \sum_{n=1}^{\infty} \frac{f_n(z)}{\zeta_n} \exp \left[ \frac{\alpha(L - z)}{2} - \mu_n t \right], \quad (6)$$

where  $f_n(z) = \sin(\lambda_n L) \sin(\lambda_n z)$ ,  $\zeta_n = 2\alpha + L\alpha^2 + 4L\lambda_n^2$ ,  $\mu_n = D \left( \lambda_n^2 + \frac{\alpha^2}{4} \right)$ , and  $\lambda_n$  represents the  $n^{\text{th}}$  positive root of

$$\sin(\lambda L) + \frac{2\lambda}{\alpha} \cos(\lambda L) = 0, \quad (7)$$

and  $\Phi_{s, q_1}(z)$  is evaluated from Eq. (5) with  $q = q_1$ .

To represent water uptake through a uniform root architecture with root depth  $L_1$ , a piecewise uniform sink term  $S(z)$  is introduced:

$$S(z) = \begin{cases} 0, & -L \leq z \leq -L_1, \\ S_0, & -L_1 \leq z \leq 0. \end{cases} \quad (8)$$

The steady-state solution with root uptake,  $\Phi_s(z)$ , follows the piecewise formulation defined in Yuan and Lu (2005). Because the sink term is time-independent, the transient solution maintains the same eigenfunctions as the bare soil case in Eq. (6), using the uptake-adjusted steady-state  $\Phi_{s, q_1}(z)$  as the asymptotic limit. In both scenarios, the state variables  $\psi$  and  $\theta$  are subsequently inferred by inverting Eq. (2b) and (4).

### 3. PINN and P-PINN Frameworks

The framework proposed herein utilises a densely connected Feed-forward Neural Network to map an input vector  $\mathbf{x}$  to a prediction  $\hat{u}$ . Information propagates through  $L$  hidden layers via a linear transformation followed by a non-linear activation:

$$\mathbf{h}^{[l]} = \phi(\mathbf{W}^{[l]}\mathbf{h}^{[l-1]} + \mathbf{b}^{[l]}), \quad (9)$$

where  $\mathbf{W}$  and  $\mathbf{b}$  represent the weight matrices and bias vectors, respectively. The final prediction is obtained via a linear transformation at the output layer:

$$\hat{u} = \mathbf{W}^{[L]}\mathbf{h}^{[L-1]} + \mathbf{b}^{[L]}. \quad (10)$$

The training process aims to arrive at an optimal set of parameters  $\boldsymbol{\xi} = \{\mathbf{W}, \mathbf{b}\}$ , which minimises a composite loss function  $\mathcal{L}(\boldsymbol{\xi})$ . The parameters are updated iteratively using a gradient descent, where gradients are computed

using Automatic Differentiation (Brunton and Kutz, 2022).

### 3.1. Network Architectures

To achieve generalisation across a continuous range of soil hydraulic properties and boundary conditions, a Parametrised PINN (P-PINN) was implemented, inspired by the high-dimensional mapping strategies of Liu et al. (2024). A schematic diagram is presented in Figure 1.

The input vector  $\mathbf{x} \in \mathbb{R}^7$  incorporates both spatio-temporal coordinates and physical parameters:

$$\mathbf{x} = [z, t, \alpha, K_s, \theta_r, \theta_s, q_B]^\top \quad (11)$$

Sobol sequence is utilised for sampling to uniformly fill this high-dimensional domain, as it has been shown to outperform Grid and Latin Hypercube sampling in PINN convergence (Wu and Gao, 2022). Inputs are standardised via Z-score normalisation (Xu et al., 2025) to scale parameters for more accurate learning.

To prevent non-physical results and numerical instability (e.g., infinitely growing weights during ponding), the raw network output is transformed through a bias-shifted exponential mapping (Bandai and Ghezzehei, 2022a):

$$\hat{\psi} = -\exp(s \cdot \text{NN output}) + \beta, \quad (12)$$

where  $\beta = 1$ . This ensures the predicted pressure head  $\hat{\psi}$  remains physically grounded within the suction range (Bandai and Ghezzehei, 2021). While  $s$  is a learnable parameter which tunes the magnitude of the NN output.

The P-PINN operates as an unsupervised solver by minimising the RRE residual  $\mathcal{R}$ . The network output is substituted into Gardner’s constitutive model, and automatic differentiation is used to compute the spatial and temporal derivatives:

$$\mathcal{R} = \frac{\partial \theta}{\partial t} - \frac{\partial}{\partial z} \left[ K(\psi) \left( \frac{\partial \psi}{\partial z} + 1 \right) \right]. \quad (13)$$

Training is optimised through adaptive optimisation strategies such as Residual-based Adaptive Refinement (RAR) (Lu et al., 2021), and Principled Loss Function (PLF) (Chen et al., 2023), which will be presented in Sec. 3.3 and Sec. 3.4, respectively.

### 3.2. Parametrisation of Input Space

The preprocessing pipeline starts by specifying the multidimensional input space for all the inputs. Followed by sampling, normalisation and offloading to the GPU as illustrated in Figure 2.

Initial and upper boundary conditions were soft-imposed, i.e. satisfied by the loss term. Each initial and upper boundary condition point corresponds to a unique parameter combination, which is randomly selected, as illustrated in Figure 3a. The target IC ( $t = 0, z \in [-100, 0]$ ) is derived from the steady-state Gardner Solution, Eq. 2b under a pre-rainfall flux  $q_A$  (See Yuan and Lu (2005) and Ng et al. (2015) for further details). The target upper BC ( $z = 0, t \in [0, 60]$  hours) is assigned a unique set of hydraulic parameters and rainfall flux  $q_B$ . The lower boundary condition was hard imposed, i.e. satisfied by architecture, since it is fixed (Dirichlet Condition) at  $\psi = 0$ . By

defining the output as in [Lagaris et al. \(1998\)](#):

$$\psi(t, z; \mathbf{p}) = \psi_{\text{lb}} + (z - z_{\text{min}})\hat{\psi}(t, z; \mathbf{p}). \quad (14)$$

The collocation points were sampled by decoupling physical parameters from spatio-temporal coordinates. The scenarios are sampled using the Sobol quasi-random sequence. The 2D spatio-temporal coordinates are tiled (identical grid) for each scenario via a 2D Sobol sequence, as illustrated in Figure 3b. Consequently, the network learns the effect of different hydraulic parameter sets at the exact position in time and space.

All sampled parameter vectors must satisfy two physical constraints to avoid undesired physical scenarios. The first scenario enforces  $\theta_s > \theta_r$ , since a non-positive porosity ( $\theta_s - \theta_r$ ) is physically meaningless. The second ensures  $K_s + q_B > 0$ , avoiding ponding, as it lies outside the scope of this study.

For IC and BC points, physical violations are rejected and resampled, as each point represents an independent scenario. But for the collocation points, to avoid disruptions caused by the rejection process, the values are adjusted to match the physical constraint as follows:

$$\begin{aligned} \theta_s &\leftarrow \max(\theta_r, \theta_s^{\min}) & \text{if } \theta_s \leq \theta_r, \\ q_B &\leftarrow \text{clip}(-K_s, [q_B^{\min}, q_B^{\max}]) & \text{if } K_s + q_B \leq 0. \end{aligned} \quad (15)$$

### 3.3. Adaptive Optimisation Strategies

The critical component that impacts the efficacy of learning is the choice of the non-linear activation function. Although piecewise linear functions like

ReLU are common in deep learning (LeCun et al., 2015), they are avoided in this study because their second derivatives are zero, making them unsuitable for the high-order derivatives required by the RRE. Moreover, the Sigmoid  $1/(1+e^{-x})$  was also avoided because its non-zero mean (0.5) leads the network into saturation (i.e. gradients vanish), hence preventing it from learning (Glorot and Bengio, 2010). Instead, Softsign  $x/(1 + |x|)$  and hyperbolic tangent were considered, since they mitigate this with a zero-centered profile as shown in Figures 4a, c.

A suitable choice proposed by Jagtap et al. (2020) is the Layer-wise Locally Adaptive Activation Function (L-LAAF), because it introduces a learnable scalar parameter  $a$  for each layer, allowing the network to adjust the steepness of the activation function during training (alongside the weights), as shown in Figure 4b. This mitigates the network from potentially getting stuck in local minima (Jagtap et al., 2020). Consequently, L-LAAF with tanh was adopted in this work, in a similar fashion to Bandai and Ghezzehei (2022b), where the output  $h_i^k$  is:

$$h_i^k = \tanh \left( n \cdot a_i^k \cdot z_i^k \right), \quad i = 1, \dots, d_k, \quad (16)$$

where  $z_i^k$  is the weighted sum,  $n$  is a fixed scaling factor, and  $a_i^k > 0$  are learnable scaling parameters for each neuron, initialised to 1.0. The iterator  $i$  runs from 1 to the total number of neurons in that specific layer (denoted by  $d_k$ ), where  $k$  is the number of hidden layers in the network. It was noted by Jagtap et al. (2020) that a small  $n$  slows convergence, while a value of  $n = 10$  was efficient in speeding up PINN convergence, since it scales  $a$  and that speeds up the topology change of the loss function.

The second adaptive technique is the Residual-based Adaptive Refinement (RAR), which was suggested by [Bandai and Ghezzehei \(2021\)](#) for future studies. The RAR identifies points with high residual error, and then adds extra collocation points to these areas, which in turn improve the accuracy in the wetting front and high-gradient regions ([Lu et al., 2021](#)). Applying the standard RAR to P-PINN may result in a temporal clustering of the newly added points in fewer regions rather than being spread across the whole time-domain, which may lead to undersampling of later temporal stages.

For this reason, the standard RAR is adjusted such that the temporal domain  $[t_{\min}, t_{\max}]$  is divided into  $B$  uniform intervals, i.e., for  $B = 4$ , the layers are  $[0-15h]$ ,  $[15-30h]$ ,  $[30-45h]$ ,  $[45-60h]$ . The adjusted RAR treats each layer separately, such that it identifies the number of points with high-residuals within each layer, then adds extra collocation points therein based on the top  $K$  points.

### 3.4. Composite Loss Function

The principled loss function (PLF) is an uncertainty weighting strategy that was brought forward in PINNs for the RRE by [Chen et al. \(2023\)](#). Instead of manually tuning weights for IC, BC and PDE losses, the model learns the optimal weighting parameters  $\sigma_j$ , allowing the network to balance the competing loss terms from dominating one another.

The total loss is defined as the sum of the PLF losses for  $\mathcal{R}$  (PDE), IC, and BC, (and data is optional) as in [Chen et al. \(2023\)](#):

$$\mathcal{L}_{total} = \sum_{j \in \{\text{PDE, IC, BC, data}\}} \left[ \frac{1}{2\sigma_j^2} \text{MSE}(r_j) + \ln(\sigma_j^2 + 1) \right], \quad (17)$$

where  $r_j$  represents residual losses for the PDE, IC, BC and data.  $\sigma_j$  is the learnable uncertainty weights and  $\text{MSE}(r_j)$  is the mean squared residual and is defined as:

$$\text{MSE}(r_j) = \frac{1}{N_j} \sum (r_j)^2. \quad (18)$$

The first term in Eq. (17) penalises the weight of the residual based on its magnitude, while the second term of the same equation prevents the weights from exploding (Chen et al., 2023).

By expanding  $r_j$  in Eq. (17), for each of the losses, yields:

$$r_{\text{PDE}}(z, t) = \frac{\partial \hat{\theta}}{\partial t} - \frac{\partial}{\partial z} \left[ \hat{K} \left( \frac{\partial \hat{\psi}}{\partial z} + 1 \right) \right] - S, \quad (19a)$$

$$r_{\text{IC}}(z, 0) = \hat{\theta}(z, 0) - \theta_{\text{IC}}(z), \quad (19b)$$

$$r_{\text{BC}}^{\text{top}}(0, t) = -\hat{K} \left( \frac{\partial \hat{\psi}}{\partial z} \Big|_{z=0} + 1 \right) - q_B, \quad (19c)$$

$$r_{\text{data}}(z_i, t_i) = \hat{\theta}(z_i, t_i) - \theta_{\text{obs},i}. \quad (19d)$$

## 4. Numerical Experiments

### 4.1. Benchmark Problems

Analytical solutions for 1D flow in unsaturated soil (Yuan and Lu, 2005) were used to validate the standard PINNs and P-PINNs models' predictions and support the standard PINN training with sparse measurement points. The analytical solutions were computed for both bare soil and uniform root uptake scenarios. The analytical solution dataset comprises:

1. Baseline configuration (Table 2): it defines the spatio-temporal domain, boundary conditions, and the Gardner hydraulic parameters used by

[Yuan and Lu \(2005\)](#), which will be used for validating the standard PINN predictions.

2. Sensitivity scenarios (Table [A.1](#)): It keeps the domain  $(t, z)$  and IC ( $q_A$ ) from the baseline but perturbs one hydraulic parameter at a time to evaluate the P-PINNs' generalisation ability (the full suite of perturbation scenarios is provided in the Appendix).

#### 4.2. Training Space for P-PINN

The parameter ranges defined in Table [1](#) represent the space that is standardised over which the network is trained. The ranges were chosen to cover a wide range of soils from fine-textured ( $\sim$  loams) to coarse-textured ( $\sim$  sandy loams) soils ([Reynolds and Elrick, 1987](#)). The target scenarios are near saturation.

The wide range of the shape parameter  $\alpha$  allows the model to learn physics from a strong capillary diffusion to advection-dominated flows ([Zhang et al., 2016](#)). Also, the high infiltration rate causes sharp wetting fronts, which are known to not only challenge classical numerical schemes but also PINNs ([Bandai and Ghezzehei, 2022b](#)). For this reason, adaptive strategies (as presented in Sec. [3.3](#)) were implemented.

#### 4.3. Performance Metrics

Table [6](#) in the supplementary tables presents a summary of the evaluation metrics used in quantifying the models' performance. Global accuracy is measured using MAE and RMSE, while the pointwise percent relative error was used to visualise the spatio-temporal distribution of discrepancies in contour plots. Given the sensitivity of  $R^2$  to outliers, the Nash-Sutcliffe

Efficiency (NSE) for the P-PINN was used to better capture the deviations from 1:1 ground-truth line (Moriassi et al., 2007). The PBIAS determines the systematic over- or under-estimation.

#### 4.4. Training Setup

A global random seed of 42 was applied to all modules to ensure consistency across runs. Both models follow a hybrid optimisation strategy as brought up by (Raissi et al., 2019), and has been widely used for PINNs for the RRE since then, as presented in Figure 5. The training starts by initialisation of the network weights  $\mathbf{W}$  using Xavier initialisation, to avoid vanishing gradients. While the bias parameters  $b$  were initialised to 0 (Glorot and Bengio, 2010; Paszke et al., 2019). The PLF parameter  $\sigma$  was initialised to 0.8.

Both models have an early stopping mechanism (patience) of 1000 iteration, as inspired by Chen et al. (2023), which terminates the training if the losses do not improve after 1000 iterations. Albeit both models share the similar foundational network and configurations, there are key differences to account for the added dimensionality and complexity of the P-PINN, which are presented in Table 3.

#### 4.5. Implementation in PyTorch

The implementation of the P-PINN models in PyTorch focused on hardware accelerated execution (Paszke et al., 2019). Computing the residual requires computing temporal derivatives and second-order spatial derivative  $\partial^2\psi/\partial z^2$ , as can be inferred from Figure 1, PyTorch’s `autograd` engine is used to compute the required derivatives efficiently. It does so by preserving

the computational graph effectively (using `create_graph=True`). Thus, allowing the network prediction to be differentiable with respect to the spatial coordinates during the loss calculation. As can be inferred from Figure 1, there is a normalisation process, but no denormalisation, and that is because the normalisation is handled via the AD chain rule, hence any scaling done there will be scaled back to the physical domain (Paszke et al., 2019, 2017), i.e.

$$\frac{\partial \psi}{\partial z} = \frac{1}{\sigma_z} \frac{\partial \psi_{\text{norm}}}{\partial z_{\text{norm}}}. \quad (20)$$

For expediting the training effectively, the pipeline uses a hybrid device strategy between CPU and GPU, which assigns operations based on their computational characteristics, as illustrated in Figure 6. Computationally intensive operations such as the forward pass (incl. General Matrix Multiplication, AD) and back propagation are offloaded to the Device GPU, since GPUs are throughput-oriented (Soyata, 2018). On the other hand, the data sampling remains on the host (CPU, as PyTorch’s `SobolEngine` is implemented only to run on CPUs (Paszke et al., 2019), due to the sequential nature of the Sobol sequence. The CPU is latency-oriented and can accelerate serial computations using techniques like prefetching, pipelining, and caches (Hager and Wellein, 2010).

The main bottleneck in this architecture is the limited amount of data that can be transferred between the CPU and GPU per unit time (bandwidth) of the bridge connecting the CPU and GPU, PCIe bus (Soyata, 2018). To reduce this bottleneck, data transfer through the PCIe bus has to be minimised. This is done by offloading the training set once, labelled as “1. Initial Transfer” in Figure 6. In addition, RAR operations (sampling, filtering, and

new points creation) are carried out directly on the GPU tensors, which is illustrated by a red loop in Figure 6.

Moreover, managing synchronisation points between the host and the device was also taken into account. Two operations lead to synchronisation barriers, which block the CPU from offloading more kernels, which in turn may lead to GPU stalling. One is logging and monitoring, labelled as “2. Sync: loss.item()” in Figure 6, using the `.item()` to extract loss values, which forces the CPU to wait for the specific GPU execution (kernel) to finish. The second operation is checkpointing, labelled as “3. Checkpoint I/O” in the same Figure, which is responsible for transferring weights from the GPU to the CPU before writing to disk. Since the latter has a bigger impact than the former, checkpointing is performed once every 1000 iterations, while logging is performed every 100 iterations.

## 5. Results

This section presents a quantitative evaluation of PINNs and P-PINNs and compares their performance against analytical solutions. It begins by assessing the baseline standard PINN under bare soil conditions, with and without sparse data points, and subsequently extends the analysis to uniform root uptake scenarios. The optimisation of the P-PINN architecture and its generalisation performance across the parameter space  $\mathcal{P}$  are then presented. Finally, a comparative analysis of computational cost is presented.

### 5.1. PINNs for Bare Soil

Figure 7a illustrates the stiff behaviour of the training dynamics of the unsupervised PINN for bare soil over  $1.3 \times 10^4$  iterations. The total loss

decreased by approximately two orders of magnitude until the early stopping criteria were met (a patience of  $10^3$  iterations) at  $\mathcal{L} = 1.16 \times 10^{-3}$ . The loss plot shows high-frequency oscillations of the competing loss terms (PDE  $\mathcal{R}$ , IC, and upper BC). Figure 7b shows the adaptive noise parameters ( $\sigma$ ) of the PLF plunging from an initial value of 0.80 to final values of  $[0.01, 0.003, 0.02]$  for the IC (steady state), Upper BC (Neumann), and PDE  $\mathcal{R}$  components, respectively.

The predicted, analytical solutions and the percentage relative error, for the two state variables, are presented in Figures 8a–c for  $\theta$  and in Figures 8d–f for  $\psi$ , respectively. The spatial domain (y-axis) spans depth from the soil surface to the water table  $z \in [0, -100]$  cm, while the x-axis spans the temporal domain  $t \in [0, 60]$  h.

Overall performance shows no significant discrepancies between the predicted solutions in Figures 8a, d and the ground truth in Figures 8b, e. By observing the relative absolute error plot in Figures 8c, f, the mean relative error for  $\theta$  is 0.085%, while for  $\psi$  is 3.600%. Meanwhile, the maximum errors exceeding the 95<sup>th</sup> percentile are localised near the initial condition and upper boundary in both Figures. However, an extra peak is observed in the period of 25 – 35 hours.

For  $\theta$ , a noticeable error starts at the soil surface until a depth of  $-70$  cm, then drops as it reaches the water table. While for  $\psi$ , the error starts from the water table (bottom BC), till  $-30$  cm, then decreases as it reaches the soil surface. Two additional peaks in relative error (See Figure 8c) were observed with  $\theta$  prediction, which is observed at the IC near the upper BC, and another around the 10<sup>th</sup> hour from the upper BC to  $-60$ cm depth. Furthermore, both

plots observe errors in the period of 55 – 60 hours.

The standard PINN model (Physics-only) shows strong alignment with the analytical solution in Figure 9. The model captures the sharp gradients at the wetting front and the water content throughout the soil. However, minor discrepancies appear as a phase error in the temporal progression. At  $t = 10$  h, the model overestimates the wetting front depth, while at  $t = 30$  h, a slight phase shift is clear. Moreover, these errors appear clustered at the steep wetting front as previously observed in Figure 9.

A quantitative comparison of the PINN performance across training scenarios involving measurement points is presented in Table 4.

The Physics-Only baseline model has recorded the highest error rates ( $\text{RMSE}_\theta = 3.77 \times 10^{-4}$  and  $\text{RMSE}_\psi = 0.173$ ), yet still maintaining high  $R^2 = 0.9999$ .

Introducing sparse measurement data (5 equidistant points across the vertical domain) dropped the  $\text{RMSE}_\theta$  by  $\approx 40\%$  and the  $\text{RMSE}_\psi$  by  $\approx 8\%$ . Increasing data density in the Fine  $\theta$  scenario resulted in even better results, reducing  $\text{RMSE}_\theta$  further by 30%. The results show that standard PINN can achieve high accuracy with sensor data placed 20 cm apart.

## 5.2. PINN with root uptake

The complexity was increased by including a sink term  $S(z, t)$  into the residual  $\mathcal{R}$ , accounting for a uniform root water uptake. The PINN has successfully adjusted to the new physical regime, and that is evident with  $\psi$  scale, which has more than doubled in Figure 10e when compared to the bare soil case in Figure 8e, to account for the high suction extracted by the roots.

The relative error plots in Figures 10c, f, the mean relative error for  $\theta$  is 0.553%, while for  $\psi$  is 3.061%. Although the error scale for  $\theta$  has increased in Figures 10c, f, when compared to bare soil in Figures 8c, f, the mean relative error for  $\theta$  is 0.553%, while for  $\psi$  is 3.061%, respectively. The largest errors (beyond the 95<sup>th</sup> percentile) remain clustered at the IC and near the upper BC for both state variables as before, leaving the majority of the spatio-temporal domain at much lower errors.

### 5.3. Generalisation and Sensitivity Analysis

The width of the network ( $N$ ) was prioritised over the depth of the network ( $L$ ), as demonstrated in Table 5. Increasing the depth from 4 to 6 layers at fixed width (64 neurons) yields negligible improvement in MAE (0.012053  $\rightarrow$  0.012038). However, widening the network to 128 neurons resulted in a 59% reduction in MAE (0.012038  $\rightarrow$  0.004923). Consequently, the  $6 \times 128$  architecture is selected.

Figure 11a presents NSE for both  $\theta$  (blue circles) and  $\psi$  (orange squares), where the perturbed parameter is stacked on the x-axis of Figure 11b. An ideal model has an NSE  $\approx 1.0$ . The PBIAS is presented in Figure 11b to further analyse whether each scenario underestimates or overestimates the state variables. An overprediction is a negative percentage, while an underprediction is a positive percentage.

The P-PINN showed high predictive accuracy across most of the parameter space, achieving an NSE  $> 0.99$  and PBIAS  $< 5\%$  for 19 out of 21 tested scenarios. The model showed the highest scores in predicting scenarios with changes in  $K_s$  and  $q_B$ , and their contour plots are shown in Figure 12. The mean relative error for the three scenarios in Figures 12c, f, i is

0.295%, 0.308% and 0.245%, respectively. The largest errors (beyond the 95<sup>th</sup> percentile) are primarily localised near the initial condition and the upper boundary. On the other hand, the model performance dropped in the  $\alpha = 0.01$  scenario,  $\text{NSE}_\theta \approx 0.96$ , which remarkably overestimates the suction values with  $\text{PBIAS}_\psi = -13.5$ . To further analyse this behaviour, the contour and profile plots are shown in Figures 13a–c and 14a. The error peaks from [5  $\rightarrow$  25 h] in the contour plots of Figure 13 look like phase error in the profile plots of Figure 14. Similar behaviour was observed with the  $\theta_s = 0.35$  and  $\alpha = 0.03$  scenarios.

For all the 3 cases, the error observed at the IC at the surface (upper BC) is similar to what happened previously with the standard PINN model in Figure 9a, where the steady state IC would curl towards the upper BC (overestimation of  $\theta$ ). The mean relative error occurred when  $\alpha = 0.01$  is 1.057%, for  $\theta_s = 0.35$  it is 1.120%, and for  $\alpha = 0.03$  is 0.953%. Moreover, the largest errors (beyond the 95<sup>th</sup> percentile) are primarily localised near the initial condition and the upper boundary.

#### 5.4. Computational Cost

Figure 15 compares the computational cost of the frameworks, which was carried out on a single NVIDIA L40S GPU (Ada Lovelace architecture) with  $\approx 44.4$  GB user accessible memory. The P-PINN took  $\approx 35.7$  minutes to train, that is, a  $2.3\times$  increase in computational cost when compared to the standard PINN 15.4 minutes. This increase is due to the wider search space and more diverse scenarios. Nevertheless, this training cost is a one-off; many scenarios can be evaluated using that trained model almost instantly. Unlike the standard PINN, which requires re-training for every new hydraulic

property.

Consequently, for applications requiring the evaluation of more than 2 hydraulic scenarios, the P-PINN is computationally more efficient with the setup used in this study. Despite its high training time, the unit cost per scenario becomes cheaper when evaluating various infiltration scenarios.

## 6. Discussion

The training dynamics of the standard PINN showed high frequency oscillations between the competing loss components (PDE residual  $\mathcal{R}$ , IC, and upper BC), implying a stiff behaviour, which is well documented in literature as a reason that affects convergence (Wang et al., 2022). Nevertheless, the probabilistic loss function (PLF) has reduced this issue by dynamically balancing the weights among the competing loss components, which is apparent in the drop of  $\sigma$  parameter from 0.80 to near-zero values (See Figure 7b).

Direct quantitative comparison with existing literature is limited by our larger computational domain and different hydraulic properties, and the differences between case-specific PINN versus a universal P-PINN approximator. Nevertheless, our standard PINN achieved a mean absolute error for  $\theta$  of 0.000369, which is comparable to the 0.000486 reported by Bandai and Ghezzehei (2022b). Albeit our pressure head maximum absolute error of 0.149 is higher than the 0.019 maximum absolute error observed by Chen et al. (2023), this is due to the scaling artefact of the larger domain. Moreover, we qualitatively observe similar error localisation at the initial and upper boundary conditions.

### 6.1. Spectral Bias at High-Gradient Regions

The phase errors observed at the wetting front, for both PINN and P-PINN results, are known as spectral bias, where the network accurately captures low-gradient regions but struggles to capture high-gradient regions. This behaviour was demonstrated by Wang et al. (2022) through studying the Neural Tangent Kernel (NTK), which describes how changing the network parameters affects the function output and convergence rate during gradient descent. Their results showed that convergence is quick for low-frequency components (which correspond to large eigenvalues), whereas high-frequency components have small eigenvalues, resulting in a slower convergence rate. Consequently, the optimiser becomes biased towards the low-frequency features, which in turn treats the high-frequency residuals as noise during the early training phase.

### 6.2. Effect of Network Architecture

The improvement in MAE when increasing network depth at constant width ( $4 \times 64 \rightarrow 6 \times 64$ ) is negligible when compared with the 59% error reduction that was achieved by increasing the width ( $6 \times 128$ ). This supports the study by Dashtbayaz et al. (2024), which not only demonstrated that wider networks mitigate the chances that the optimiser gets stuck in local minima. But also been proven that a wider network shows a stable training regime than a narrower model, which showed a chaotic gradient.

### 6.3. Sensitivity to Hydraulic Parameters

The P-PINN exhibited its largest errors at  $\theta_s = 0.35$ ,  $\alpha = 0.01$ ,  $0.03$  scenarios. Starting with the low shape parameter scenario  $\alpha = 0.01$ , although

it scored a lower error by 2 orders of magnitude in the standard PINN model than the P-PINN, this shows how sensitive the soil type is to the network, and how the behaviour varies non-linearly between different soil types.

In the  $\alpha = 0.01$  scenario in Figures 13a–c, errors peaked during infiltration [10 – 25 h]. A low  $\alpha$  is related to fine-textured soil, which is controlled by capillary forces, and as mentioned before, results in a numerically stiff problem. While high  $\alpha$  is controlled by advection. It seems that low  $\alpha$  results in spectral bias. This can also be connected to the causality issue in PINN training in Wang et al. (2024), where the network does not propagate the solution in the same way the classical numerical methods do by time-stepping, but rather it views the problem in space and time simultaneously, and error accumulates this way.

Shifting onto the second problematic case, that is, the low porosity scenario  $\theta_s = 0.35$ . The relative error started from the IC at the upper BC, diffused, and smeared along the spatio-temporal domain, then dropped beyond the wetting front, which is consistent with the spectral bias of neural networks. Although it captures a relatively similar profile to the analytical solution as shown in Figure 14b, it fails to resolve the precise temporal behaviour of the wetting front.

By further analysing the PBIAS plot in Figure 11b, for  $\alpha = 0.01$  and  $\theta_s = 0.35$ , the model exhibited an overestimation of the suction in both scenarios, and simultaneously an underestimation of  $\theta$ . By referring to the SWRC relationship, where a small decrease in  $\theta$  translates to a large increase in the magnitude of  $\psi$ . This increase in  $\psi$  makes the wetting profile even smoother, hence escaping the high gradient area (spectral bias).

#### 6.4. Computational Cost and Applicability

Approximately bifold increase in P-PINN training time is related to the increased parameter space and the need to learn across multiple scenarios simultaneously. However, the computational investment occurs only once during the training of P-PINN. Then, the P-PINN can evaluate multiple scenarios within the parameter space  $\mathcal{P}$  almost instantly. While the standard PINN requires complete retraining for each new parameter set. In the current study, it was shown that if more than two scenarios needs to be evaluated, the P-PINN becomes computationally advantageous. Furthermore, the addition of the sink term representing a uniform root water uptake demonstrates that the framework can be extended to more complex physical processes without extensive changes to the architecture. It should also be noted that the ratio of training time between P-PINN and standard PINN can be different with a different hardware setup.

### 7. Conclusions

PINNs have been gaining interest and being applied to various problems with scientific and practical interest since their introduction. Solving RRE with PINNs has been repetitively done at small spatio-temporal domains with fixed parameter spaces without capturing the differences in soil's response to rainfall with varying soil parameters. This study has successfully demonstrated the efficacy of PINNs in solving the forward RRE over larger spatio-temporal domains than the existing work in the literature, with high accuracy in capturing the infiltration process with an  $\text{RMSE}_\theta$  of  $3.77 \times 10^{-4}$  for training with physics only. Training with 5 sparse data points to imi-

tate realistic field sensors leads to improved accuracy by 40% in predicting water contents than without data at all. We also demonstrated that the parametrisation has enabled generalisation capabilities of PINNs for RRE, which achieved a  $NSE > 0.99$  in 19 out of 21 tested scenarios. A further contribution of this work is the addition of a root water uptake sink term into the PINN architecture, so that alterations in soil hydrological regime due to root water uptake can also be modelled. Finally, although P-PINN requires more than double the training cost of a standard PINN, its unit cost for evaluating many scenarios is cheaper than the standard PINN

### **Author contributions**

**MG:** Methodology, Investigation, Software, Formal analysis, Writing - Original draft, Visualisation: **AY:** Conceptualisation, Methodology, Writing - Review & Editing, Supervision

### **Funding**

We acknowledge the contribution of RWTH Aachen University to the open Access funding within the framework of DEAL initiative.

### **Data availability statement**

Prediction datasets used in this study can be found in [Gowely and Yildiz \(2026\)](#) published on Zenodo.

### **Declaration of generative AI use**

During the preparation of this work the first author used Google Gemini in order to generate code to facilitate the plotting process. After using this

tool/service, both authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

## Appendix A. Sensitivity Analysis Scenarios

Table A.1: Perturbation scenarios for P-PINN validation.

| Scenario               | $\alpha$ [cm <sup>-1</sup> ] | $K_s$ [cm h <sup>-1</sup> ] | $\theta_s$ [-] | $\theta_r$ [-] | $q_B$ [cm h <sup>-1</sup> ] |
|------------------------|------------------------------|-----------------------------|----------------|----------------|-----------------------------|
| — Varying $\alpha$ —   |                              |                             |                |                |                             |
| $\alpha = 0.01$        | <b>0.01</b>                  | 1.0                         | 0.45           | 0.20           | -0.9                        |
| $\alpha = 0.03$        | <b>0.03</b>                  | 1.0                         | 0.45           | 0.20           | -0.9                        |
| $\alpha = 0.07$        | <b>0.07</b>                  | 1.0                         | 0.45           | 0.20           | -0.9                        |
| $\alpha = 0.09$        | <b>0.09</b>                  | 1.0                         | 0.45           | 0.20           | -0.9                        |
| $\alpha = 0.10$        | <b>0.10</b>                  | 1.0                         | 0.45           | 0.20           | -0.9                        |
| — Varying $K_s$ —      |                              |                             |                |                |                             |
| $K_s = 1.1$            | 0.05                         | <b>1.1</b>                  | 0.45           | 0.20           | -0.9                        |
| $K_s = 1.2$            | 0.05                         | <b>1.2</b>                  | 0.45           | 0.20           | -0.9                        |
| — Varying $\theta_s$ — |                              |                             |                |                |                             |
| $\theta_s = 0.35$      | 0.05                         | 1.0                         | <b>0.35</b>    | 0.20           | -0.9                        |
| $\theta_s = 0.40$      | 0.05                         | 1.0                         | <b>0.40</b>    | 0.20           | -0.9                        |
| $\theta_s = 0.49$      | 0.05                         | 1.0                         | <b>0.49</b>    | 0.20           | -0.9                        |
| $\theta_s = 0.53$      | 0.05                         | 1.0                         | <b>0.53</b>    | 0.20           | -0.9                        |
| $\theta_s = 0.58$      | 0.05                         | 1.0                         | <b>0.58</b>    | 0.20           | -0.9                        |
| — Varying $\theta_r$ — |                              |                             |                |                |                             |
| $\theta_r = 0.10$      | 0.05                         | 1.0                         | 0.45           | <b>0.10</b>    | -0.9                        |
| $\theta_r = 0.13$      | 0.05                         | 1.0                         | 0.45           | <b>0.13</b>    | -0.9                        |
| $\theta_r = 0.16$      | 0.05                         | 1.0                         | 0.45           | <b>0.16</b>    | -0.9                        |
| $\theta_r = 0.19$      | 0.05                         | 1.0                         | 0.45           | <b>0.19</b>    | -0.9                        |
| $\theta_r = 0.22$      | 0.05                         | 1.0                         | 0.45           | <b>0.22</b>    | -0.9                        |
| $\theta_r = 0.25$      | 0.05                         | 1.0                         | 0.45           | <b>0.25</b>    | -0.9                        |
| — Varying $q_B$ —      |                              |                             |                |                |                             |
| $q_B = -0.6$           | 0.05                         | 1.0                         | 0.45           | 0.20           | <b>-0.6</b>                 |
| $q_B = -0.8$           | 0.05                         | 1.0                         | 0.45           | 0.20           | <b>-0.8</b>                 |
| $q_B = -0.9$           | 0.05                         | 1.0                         | 0.45           | 0.20           | <b>-0.9</b>                 |

## Appendix B. Training algorithm

---

### Algorithm 1: P-PINN Training

---

#### Require:

- 1: Parameter Ranges  $\mathcal{P}_{range}$  for  $\mathbf{p} = [\alpha, K_s, \theta_r, \theta_s, q_B]$
- 2: Domain  $t \in [t_{min}, t_{max}]$ ,  $z \in [z_{min}, z_{max}]$
- 3: Fixed Boundary Conditions:  $q_{top}(t) = q_B$  (from  $\mathbf{p}$ ),  $\psi_{bot}$  (Dirichlet)
- 4: Architecture:  $\mathcal{N}_\psi(t, z, \mathbf{p}; \Theta)$

#### Initialisation:

- 5: Initialise network weights  $\Theta$  and uncertainty weights  $\sigma$   $\triangleright \sigma$  is PLF related
- 6:  $X_f = \{(t, z, \mathbf{p})_i\}_{i=1}^{N_f}$   $\triangleright$  Sample set generation using Sobol
- 7:  $\theta_s \leftarrow \max(\theta_s, \theta_r + \delta)$   $\triangleright$  Ensure physical constraints  $\theta_s > \theta_r$
- 8:  $q_B \leftarrow \max(q_B, -K_s + \delta)$   $\triangleright$  Ensure no ponding:  $K_s + q_B > 0$

#### Training (RAR Loop):

- 9: **for**  $k = 1$  to  $K_{RAR}$  **do**

#### Optimisation Loop (Adam $\rightarrow$ L-BFGS):

- 10:   **for** iter  $i = 1$  to  $N_{iter}$  **do**  $\triangleright$  Forward Pass
- 11:       Sample batch  $(t, z, \mathbf{p})$  from  $X_f$   $\triangleright$  If batch size = null, all points are used
- 12:       Predict raw suction:  $\hat{\psi}_{raw} \leftarrow \mathcal{N}_\psi(t, z, \mathbf{p})$
- 13:        $\hat{\psi} \leftarrow \text{blend}(z) \cdot \hat{\psi}_{raw} + \psi_{bot}$   $\triangleright$  Hard imposed Dirichlet bottom BC Lagaris et al. (1998); Lu et al. (2021)
- 14:        $\hat{K} \leftarrow K_s \exp(\alpha \hat{\psi})$ ,  $\hat{\theta} \leftarrow \theta_r + (\theta_s - \theta_r) \exp(\alpha \hat{\psi})$   $\triangleright$  Gardner's model

#### Physics Residuals (Auto-Diff):

- 15:        $r_{PDE} \leftarrow \partial_t \hat{\theta} - \partial_z [\hat{K}(\partial_z \hat{\psi} + 1)]$   $\triangleright$  PDE Residual
- 16:        $r_{IC} \leftarrow \hat{\theta}(0, z, \mathbf{p}) - \theta_{analytical}(z, \mathbf{p})$   $\triangleright$  Soft imposed steady-state IC
- 17:        $r_{BC} \leftarrow [-\hat{K}(\partial_z \hat{\psi} + 1)]_{z=0} - q_B$   $\triangleright$  Soft imposed Neumann upper BC

#### Loss Calculation (PLF):

- 18:        $\mathcal{L}_{total} = \sum_{j \in \{PDE, IC, BC\}} \left( \frac{\text{MSE}(r_j)}{2\sigma_j^2} + \ln(\sigma_j^2 + 1) \right)$   $\triangleright$  PLF as per

Chen et al. (2023)

```
19:   Update: Minimize  $\mathcal{L}_{total}$  w.r.t  $\Theta, \sigma$   $\triangleright$  Optimisation Step
20: end for
    Adaptive Refinement (Time-layers RAR):
21:   Generate candidate set  $X_{cand}$ , and filter points therein
22:   Select points with  $|r_{PDE}| > \text{tol}$  in each time-layer
23:    $X_f \leftarrow X_f \cup X_{new}$   $\triangleright$  Update Training Points
24: end for

Ensure:
25: Surrogate Model  $\hat{\psi}(t, z, \alpha, K_s, \theta_r, \theta_s, q_B)$ 
```

---

## References

- Bandai, T., Ghezzehei, T.A., 2021. Physics informed neural networks with monotonicity constraints for richardson richards equation: Estimation of constitutive relationships and soil water flux density from volumetric water content measurements. *Water Resources Research* 57. doi:[10.1029/2020wr027642](https://doi.org/10.1029/2020wr027642).
- Bandai, T., Ghezzehei, T.A., 2022a. DD-PINNS-RRE, Dataset on Zenodo. doi:[10.5281/zenodo.6030635](https://doi.org/10.5281/zenodo.6030635).
- Bandai, T., Ghezzehei, T.A., 2022b. Forward and inverse modeling of water flow in unsaturated soils with discontinuous hydraulic conductivities using physics-informed neural networks with domain decomposition. *Hydrology and Earth System Sciences* 26, 4469–4495. doi:[10.5194/hess-26-4469-2022](https://doi.org/10.5194/hess-26-4469-2022).
- Brunton, S.L., Kutz, J.N., 2022. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. 2 ed., Cambridge University Press.
- Chahinian, N., Moussa, R., Andrieux, P., Voltz, M., 2005. Comparison of infiltration models to simulate flood events at the field scale. *Journal of Hydrology* 306, 191–214. doi:[10.1016/j.jhydrol.2004.09.009](https://doi.org/10.1016/j.jhydrol.2004.09.009).
- Chen, Y., Xu, Y., Wang, L., Li, T., 2023. Modeling water flow in unsaturated soils through physics-informed neural network with principled loss function. *Comput. Geotech.* 161, 105546. doi:[10.1016/j.compgeo.2023.105546](https://doi.org/10.1016/j.compgeo.2023.105546).
- Dashtbayaz, N.H., Farhani, G., Wang, B., Ling, C.X., 2024. Physics-informed neural networks: Minimizing residual loss with wide networks and effective activations, in: *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24)*, International Joint Conferences on Artificial Intelligence Organization. pp. 5853–5861. doi:[10.24963/ijcai.2024/647](https://doi.org/10.24963/ijcai.2024/647).
- Depina, I., Jain, S., Mar Valsson, S., Gotovac, H., 2021. Application of physics-informed neural networks to inverse problems in unsaturated groundwater flow. *Georisk: Assessment and Management of Risk for Engineered Systems and Geohazards* 16, 21–36. doi:[10.1080/17499518.2021.1971251](https://doi.org/10.1080/17499518.2021.1971251).

- Elkhadrawi, M., Ng, C., Bain, D.J., Sargent, E.E., Stearsman, E.V., Gray, K.A., Akcakaya, M., 2024. Novel physics informed-neural networks for estimation of hydraulic conductivity of green infrastructure as a performance metric by solving richards-richardson pde. *Neural Computing and Applications* 36, 5555–5569. doi:[10.1007/s00521-023-09378-z](https://doi.org/10.1007/s00521-023-09378-z).
- Farthing, M.W., Ogden, F.L., 2017. Numerical solution of richards' equation: A review of advances and challenges. *Soil Science Society of America Journal* 81, 1257–1269. doi:[10.2136/sssaj2017.02.0058](https://doi.org/10.2136/sssaj2017.02.0058).
- Gardner, W., 1958. Some steady-state solutions of the unsaturated moisture flow equation with application to evaporation from a water table. *Soil Science* 85, 228–232. doi:[10.1097/00010694-195804000-00006](https://doi.org/10.1097/00010694-195804000-00006).
- Glorot, X., Bengio, Y., 2010. Understanding the difficulty of training deep feedforward neural networks, in: Teh, Y.W., Titterton, M. (Eds.), *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, PMLR, Chia Laguna Resort, Sardinia, Italy. pp. 249–256. URL: <https://proceedings.mlr.press/v9/glorot10a.html>.
- Gowely, M., Yildiz, A., 2026. Prediction dataset of parametrised pinns for water infiltration in unsaturated soils. doi:[10.5281/zenodo.19735605](https://doi.org/10.5281/zenodo.19735605).
- Hager, G., Wellein, G., 2010. *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, Boca Raton. doi:[10.1201/EBK1439811924](https://doi.org/10.1201/EBK1439811924).
- Iverson, R.M., 2000. Landslide triggering by rain infiltration. *Water Resources Research* 36, 1897–1910. doi:[10.1029/2000wr900090](https://doi.org/10.1029/2000wr900090).
- Jagtap, A.D., Kawaguchi, K., Karniadakis, G.E., 2020. Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. *Journal of Computational Physics* 404, 109136. doi:[10.1016/j.jcp.2019.109136](https://doi.org/10.1016/j.jcp.2019.109136).
- Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S., Yang, L., 2021. Physics-informed machine learning. *Nature Reviews Physics* 3, 422–440. doi:[10.1038/s42254-021-00314-5](https://doi.org/10.1038/s42254-021-00314-5).

- Lagaris, I.E., Likas, A., Fotiadis, D.I., 1998. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks* 9, 987–1000. doi:[10.24963/ijcai.2024/647](https://doi.org/10.24963/ijcai.2024/647).
- LeCun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. *Nature* 521, 436–444. doi:[10.1038/nature14539](https://doi.org/10.1038/nature14539).
- Li, Y., Sun, Q., Fu, Y., Wei, J., 2025. Solving the richards infiltration equation by coupling physics informed neural networks with Hydrus 1D. *Sci. Rep.* 15, 18649. doi:[10.1038/s41598-025-02978-w](https://doi.org/10.1038/s41598-025-02978-w).
- List, F., Radu, F.A., 2016. A study on iterative methods for solving Richards' equation. *Computational Geosciences* 20, 341–353. doi:[10.1007/s10596-016-9566-3](https://doi.org/10.1007/s10596-016-9566-3).
- Liu, K., Luo, K., Cheng, Y., Liu, A., Li, H., Fan, J., Balachandar, S., 2024. Parameterized physics-informed neural networks (p-pinns) solution of uniform flow over an arbitrarily spinning spherical particle. *International Journal of Multiphase Flow* 180, 104937. doi:[10.1016/j.ijmultiphaseflow.2024.104937](https://doi.org/10.1016/j.ijmultiphaseflow.2024.104937).
- Lu, L., Meng, X., Mao, Z., Karniadakis, G.E., 2021. Deepxde: A deep learning library for solving differential equations. *SIAM Review* 63, 208–228. doi:[10.1137/19M1274067](https://doi.org/10.1137/19M1274067).
- Moriasi, D.N., Arnold, J.G., Van Liew, M.W., Bingner, R.L., Harmel, R.D., Veith, T.L., 2007. Model evaluation guidelines for systematic quantification of accuracy in watershed simulations. *Transactions of the ASABE* 50, 885–900. doi:[10.13031/2013.23153](https://doi.org/10.13031/2013.23153).
- Ng, C., Liu, H., Feng, S., 2015. Analytical solutions for calculating pore-water pressure in an infinite unsaturated slope with different root architectures. *Canadian Geotechnical Journal* 52, 1981–1992. doi:[10.1139/cgj-2015-0001](https://doi.org/10.1139/cgj-2015-0001).
- Paszke, A., Gross, S., Chintala, S., Chanan, G., Yang, E., DeVito, Z., Lin, Z., Desmaison, A., Antiga, L., Lerer, A., 2017. Automatic differentiation in pytorch, in: *NIPS-W*.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang,

- E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. PyTorch: an imperative style, high-performance deep learning library. Curran Associates Inc., Red Hook, NY, USA. doi:[10.48550/arXiv.1912.01703](https://doi.org/10.48550/arXiv.1912.01703).
- Raissi, M., Perdikaris, P., Karniadakis, G.E., 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics* 378, 686–707. doi:[10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045).
- Reynolds, W.D., Elrick, D.E., 1987. A laboratory and numerical assessment of the guelph permeameter method. *Soil Science* 144, 282–299.
- Richards, L.A., 1931. Capillary conduction of liquids through porous mediums. *Physics* 1, 318–329. doi:[10.1063/1.1745010](https://doi.org/10.1063/1.1745010).
- Sharma, R., Rallapalli, S., Magner, J., 2025. Optimizing water-efficient agriculture: evaluating the sustainability of soil management and irrigation synergies using fuzzy extent analysis. *Scientific Reports* 15. doi:[10.1038/s41598-025-15426-6](https://doi.org/10.1038/s41598-025-15426-6).
- Soyata, T., 2018. GPU Parallel Program Development Using CUDA. Taylor and Francis.
- Srivastava, R., Yeh, T.C.J., 1991. Analytical solutions for one-dimensional, transient infiltration toward the water table in homogeneous and layered soils. *Water Resources Research* 27, 753–762. doi:[10.1029/91WR00624](https://doi.org/10.1029/91WR00624).
- Van Genuchten, M.T., 1980. A closed form equation for predicting the hydraulic conductivity of unsaturated soils. *Soil Science Society of America Journal* 44, 892–898. doi:[10.2136/sssaj1980.03615995004400050002x](https://doi.org/10.2136/sssaj1980.03615995004400050002x).
- Vanderborght, J., Leitner, D., Schnepf, A., Couvreur, V., Vereecken, H., Javaux, M., 2024. Combining root and soil hydraulics in macroscopic representations of root water uptake. *Vadose Zone Journal* 23, e20273. doi:[10.1002/vzj2.20273](https://doi.org/10.1002/vzj2.20273).
- Wang, S., Sankaran, S., Perdikaris, P., 2024. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering* 421, 116813. doi:[10.1016/j.cma.2024.116813](https://doi.org/10.1016/j.cma.2024.116813).

- Wang, S., Teng, Y., Perdikaris, P., 2022. When and why pinns fail to train: A neural tangent kernel perspective. *Journal of Computational Physics* 449, 110768. doi:[10.1016/j.jcp.2021.110768](https://doi.org/10.1016/j.jcp.2021.110768).
- Wu, C., Gao, Z., 2022. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Journal of Computational Physics* 468, 111510. doi:[10.1016/j.jcp.2022.111510](https://doi.org/10.1016/j.jcp.2022.111510).
- Wu, L., Huang, R., Li, X., 2020. *Hydro-Mechanical Analysis of Rainfall-Induced Landslides*. Springer and Science Press, Beijing. doi:[10.1007/978-981-15-3282-1](https://doi.org/10.1007/978-981-15-3282-1).
- Xu, S., Dai, Y., Yan, C., Sun, Z., Huang, R., Guo, D., Yang, G., 2025. On the preprocessing of physics-informed neural networks: how to better utilize data in fluid mechanics. *Journal of Computational Physics* 528, 113837. doi:[10.1016/j.jcp.2025.113837](https://doi.org/10.1016/j.jcp.2025.113837).
- Yuan, C., Qin, C., Li, L., Lei, X., 2025. Transient State Analysis of Rainfall Infiltration Into Layered Vegetated Soils. *International Journal for Numerical and Analytical Methods in Geomechanics* 49, 4248–4257. doi:[10.1002/nag.70068](https://doi.org/10.1002/nag.70068).
- Yuan, F., Lu, Z., 2005. Analytical solutions for vertical flow in unsaturated, rooted soils with variable surface fluxes. *Vadose Zone Journal* 4, 1073–1084. doi:[10.2136/vzj2005.0043](https://doi.org/10.2136/vzj2005.0043).
- Zhang, L., Li, J., Li, X., Zhang, J., Zhu, H., 2016. *Rainfall-Induced Soil Slope Failure: Stability Analysis and Probabilistic Assessment*. 1st ed., CRC Press. doi:[10.1201/b20116](https://doi.org/10.1201/b20116).
- Zhang, Y., Zhang, Z., Ma, Z., Chen, J., Akbar, J., Zhang, S., Che, C., Zhang, M., Cerdà, A., 2018. A review of preferential water flow in soil science. *Canadian Journal of Soil Science* 98, 604–618. doi:[10.1139/cjss-2018-0046](https://doi.org/10.1139/cjss-2018-0046).

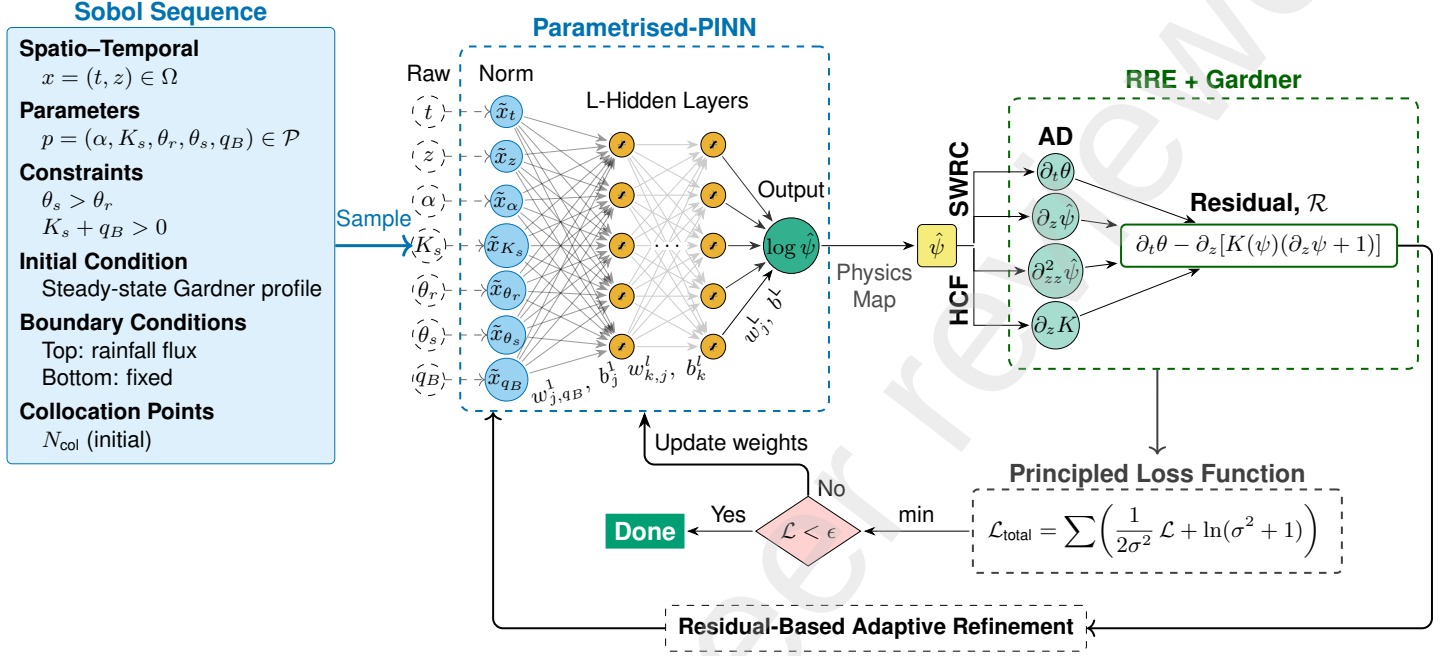


Figure 1: Schematic diagram of the P-PINN framework as inspired by Liu et al. (2024), use of physics map and constitutive relationship was inspired by Bandai and Ghezzehei (2021), PLF was inspired by Chen et al. (2023), for solving the RRE. The framework starts with Sobol sampling, a FNN with a physics-constrained mapping, AD for computing the residual using the Gardner constitutive model, PLF, and RAR adaptive functions.

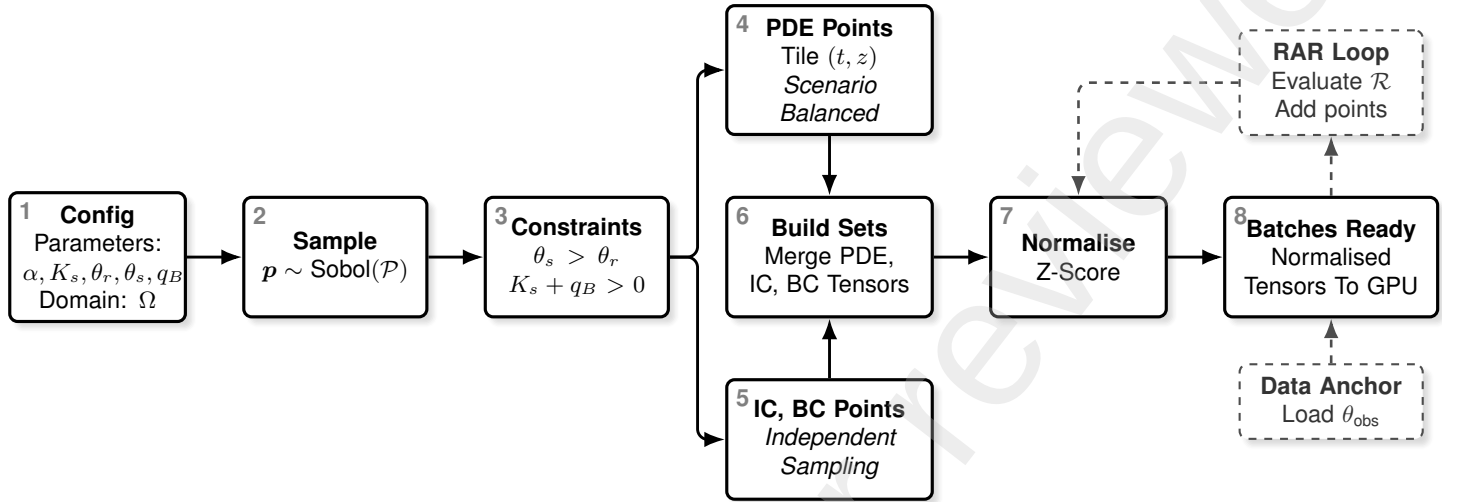


Figure 2: Schematic diagram of the P-PINN pre-processing pipeline, which transforms the hydraulic parameters and domain configuration ranges into normalised GPU tensors for training. Solid boxes represent core stages; dashed boxes denote additional components such as residual-based adaptive refinement (RAR loop) and observational data anchoring ( $\theta_{obs}$ ).

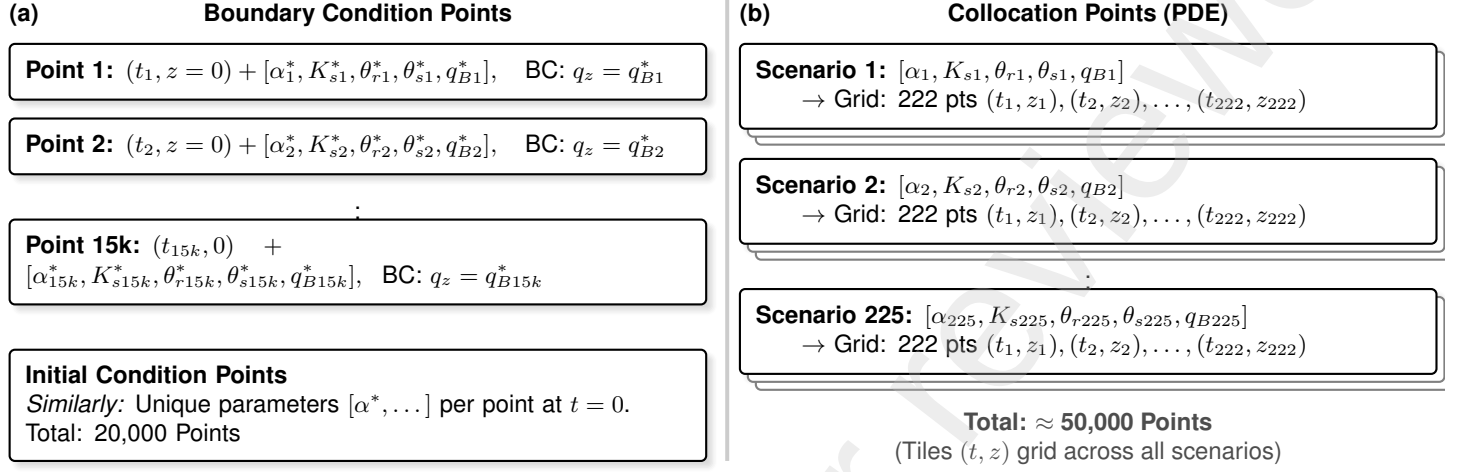


Figure 3: Schematic diagrams of the hybrid sampling strategy used for the P-PINN. (a): Boundary and initial condition points utilise a 1:1 mapping, where each spatio-temporal point is paired with a unique random parameter set ( $\mathbf{p}^*$ ). (b): Collocation points are sampled such that the spatio-temporal grid tiles across different parameter scenarios ( $\mathbf{p}$ ).

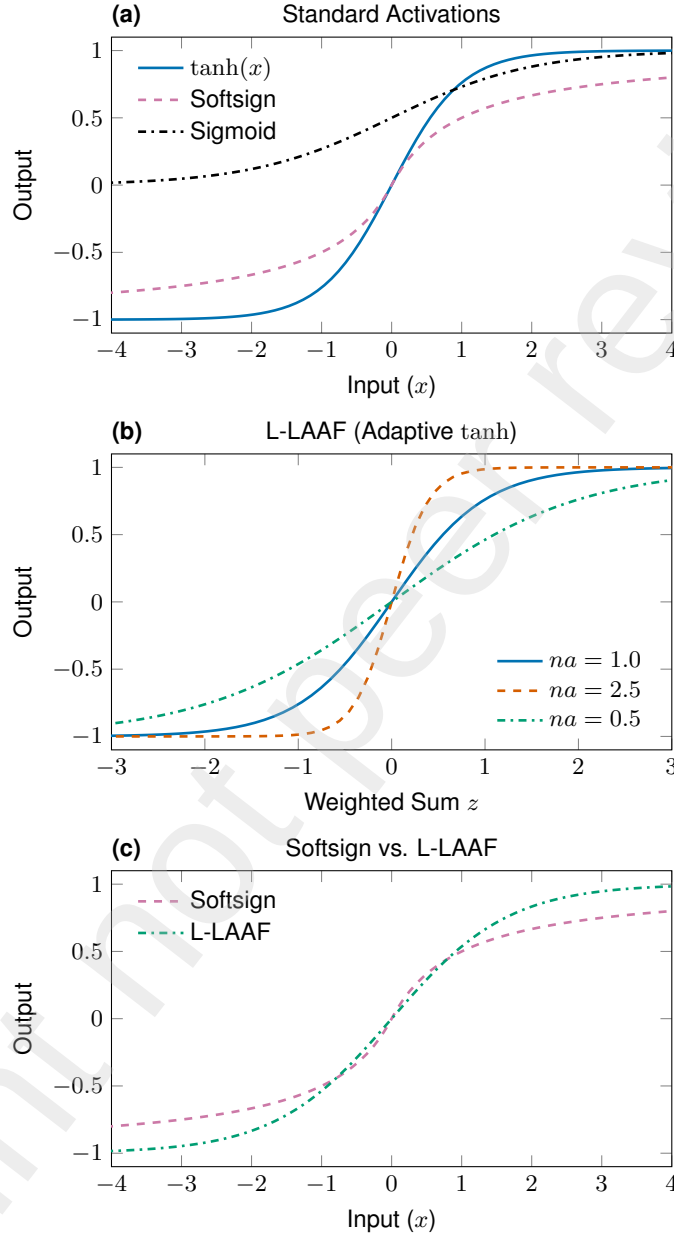


Figure 4: Comparison of activation functions. (a) Standard S-shaped activations (Tanh, Softsign, Sigmoid). (b) L-LAAF with varying slope scaling factors  $n \cdot a$  for tanh. (c) A direct comparison showing the tail behaviour of Softsign (polynomial decay) vs. adaptive tanh with  $n \cdot a = 0.5$  (exponential decay).

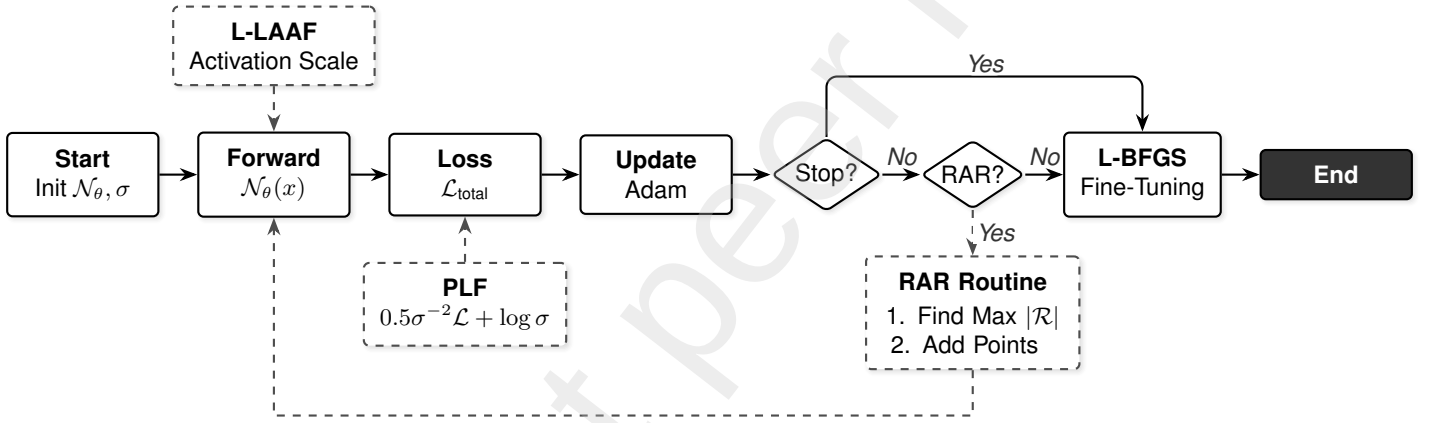


Figure 5: Flowchart of the training schedule. The training loop starts with Xavier initialisation, and forward pass to predict  $\hat{\psi}$  using L-LAAF. Then, uncertainty loss computation (PLF), Adam optimisation, and Residual-based Adaptive Refinement (RAR), before fine-tuning with L-BFGS. Dashed boxes represent the additional adaptive components (L-LAAF, PLF and RAR).

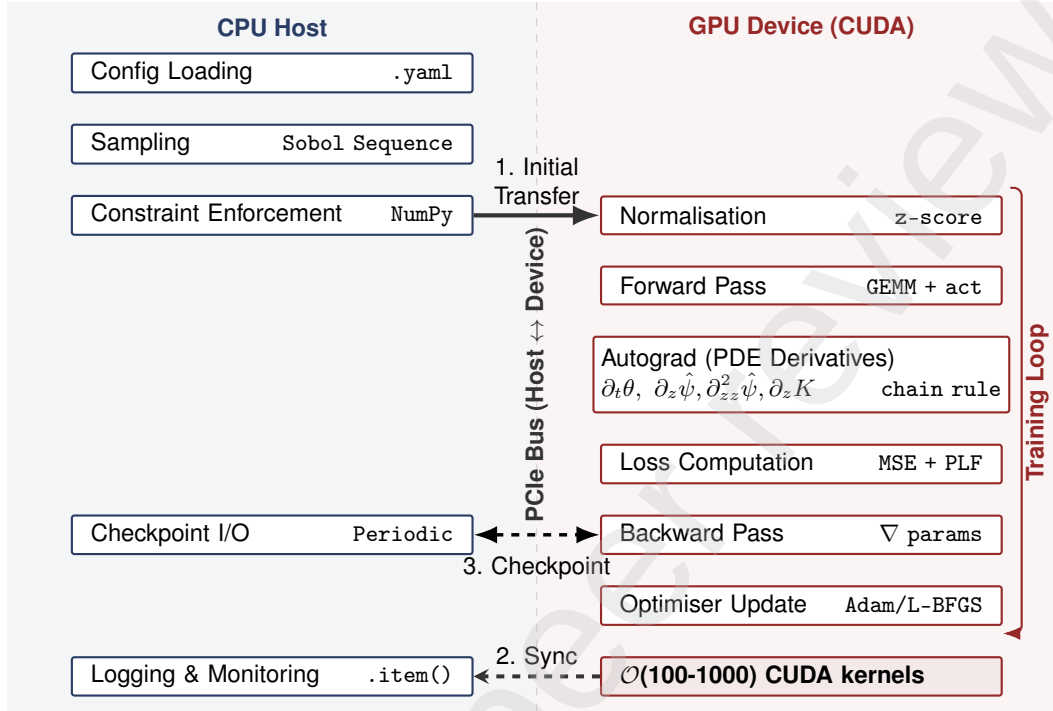


Figure 6: Data Flow between Host CPU host and GPU Device, illustrating the data management and kernel execution, training loop, along with periodic synchronisation points, denoted by dashed arrow lines, required for monitoring (labelled 2) and checkpointing (labelled 3, two arrows represent exchange state data bidirectionally).

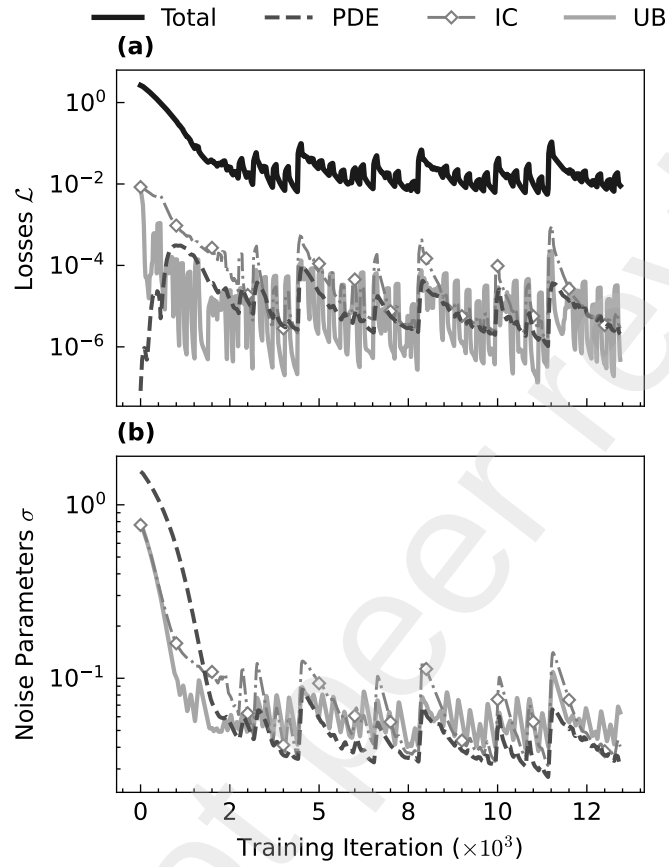


Figure 7: Training dynamics of the physics-driven standard PINN for bare soil.

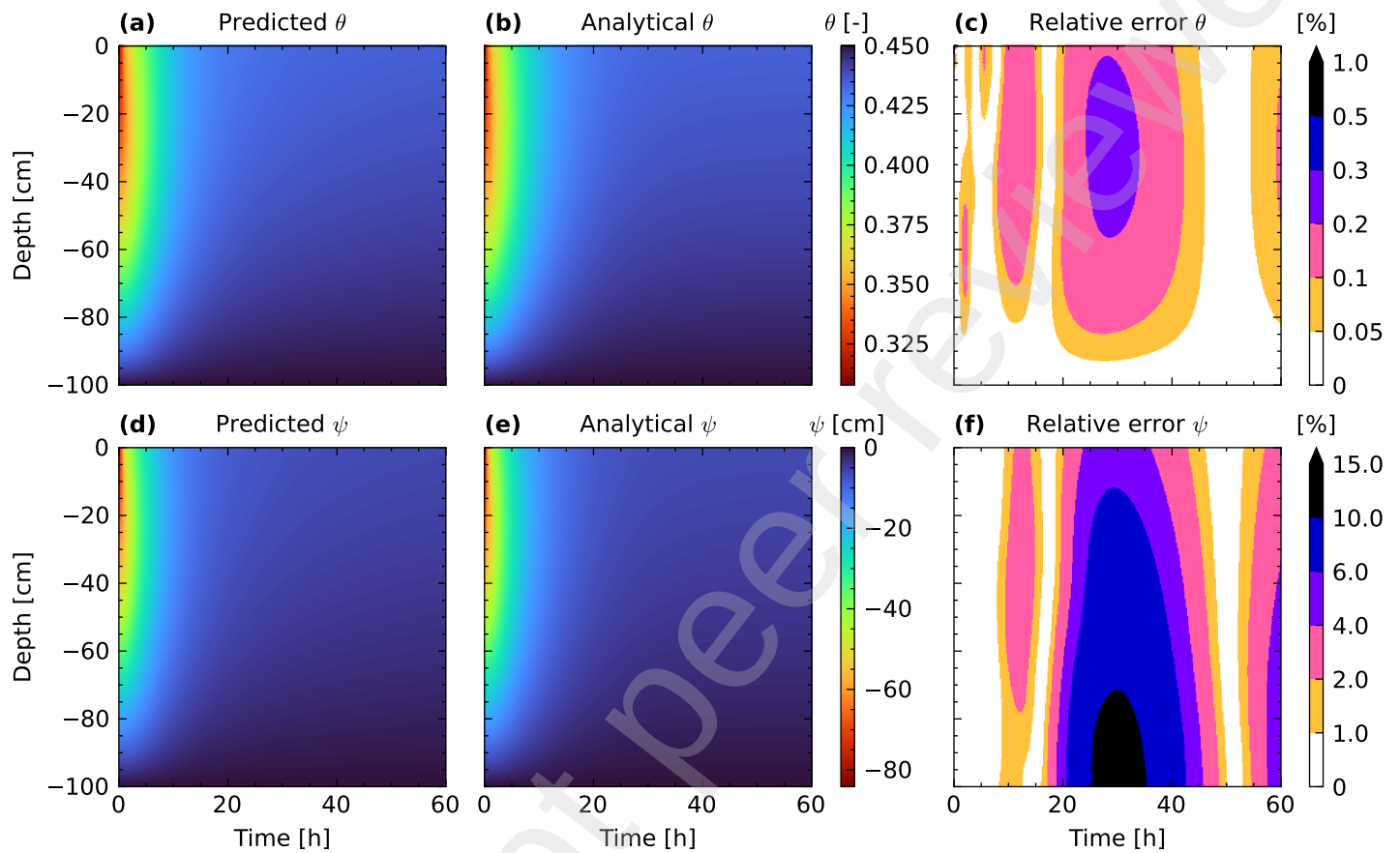


Figure 8: Spatio-temporal comparison of water content ( $\theta$ ) and pressure head ( $\psi$ ) for bare soil infiltration. PINN predictions (a, d), calculated with analytical solution (b, e), and relative error maps (c, f), respectively. Results are shown over a 60-hour simulation period across a 100 cm soil profile.

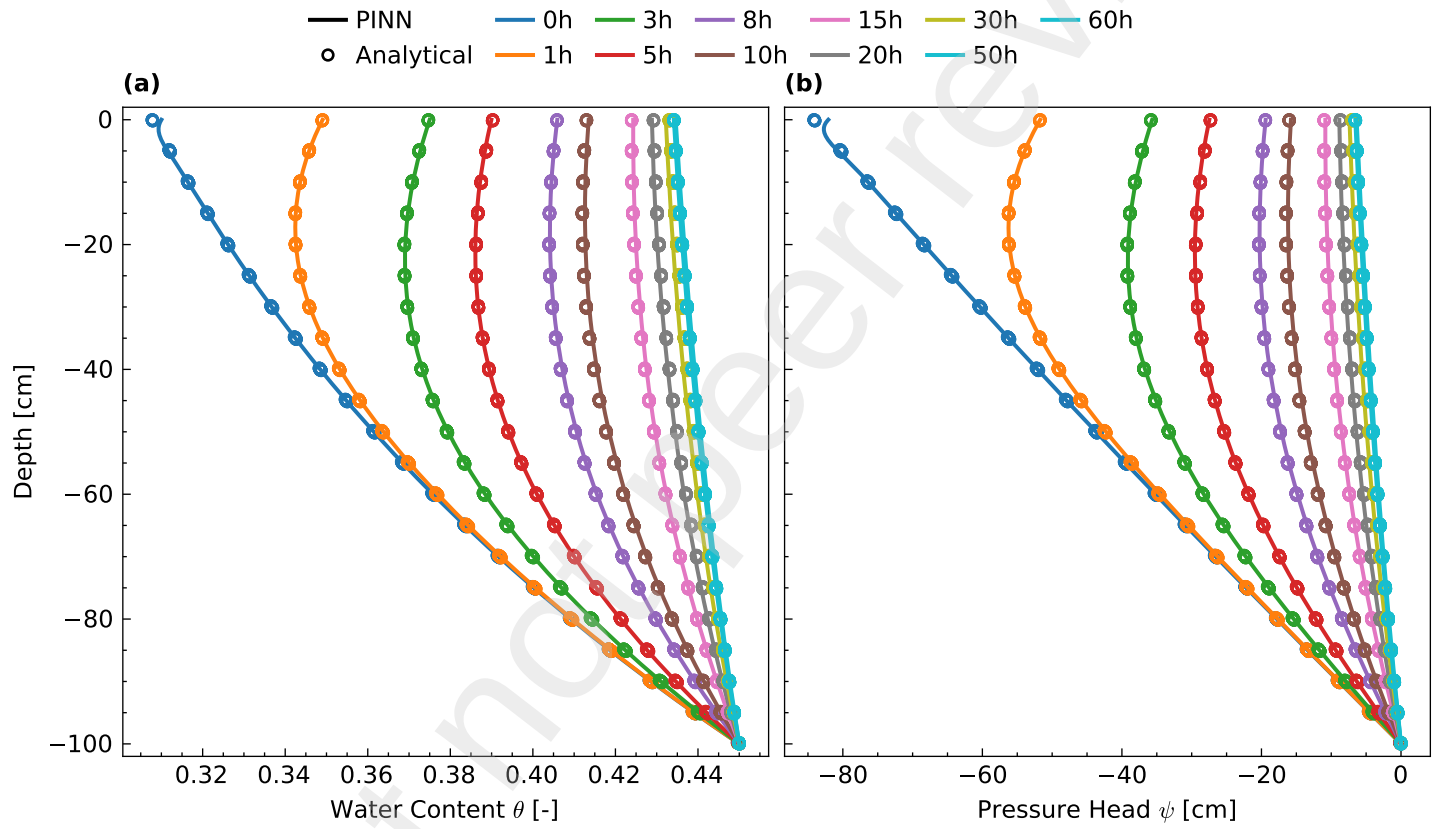


Figure 9: Vertical profiles of water content (a) and pressure head (b) at selected time snapshots for bare soil infiltration. Solid lines and open circles represent PINN predictions and analytical solutions, respectively.

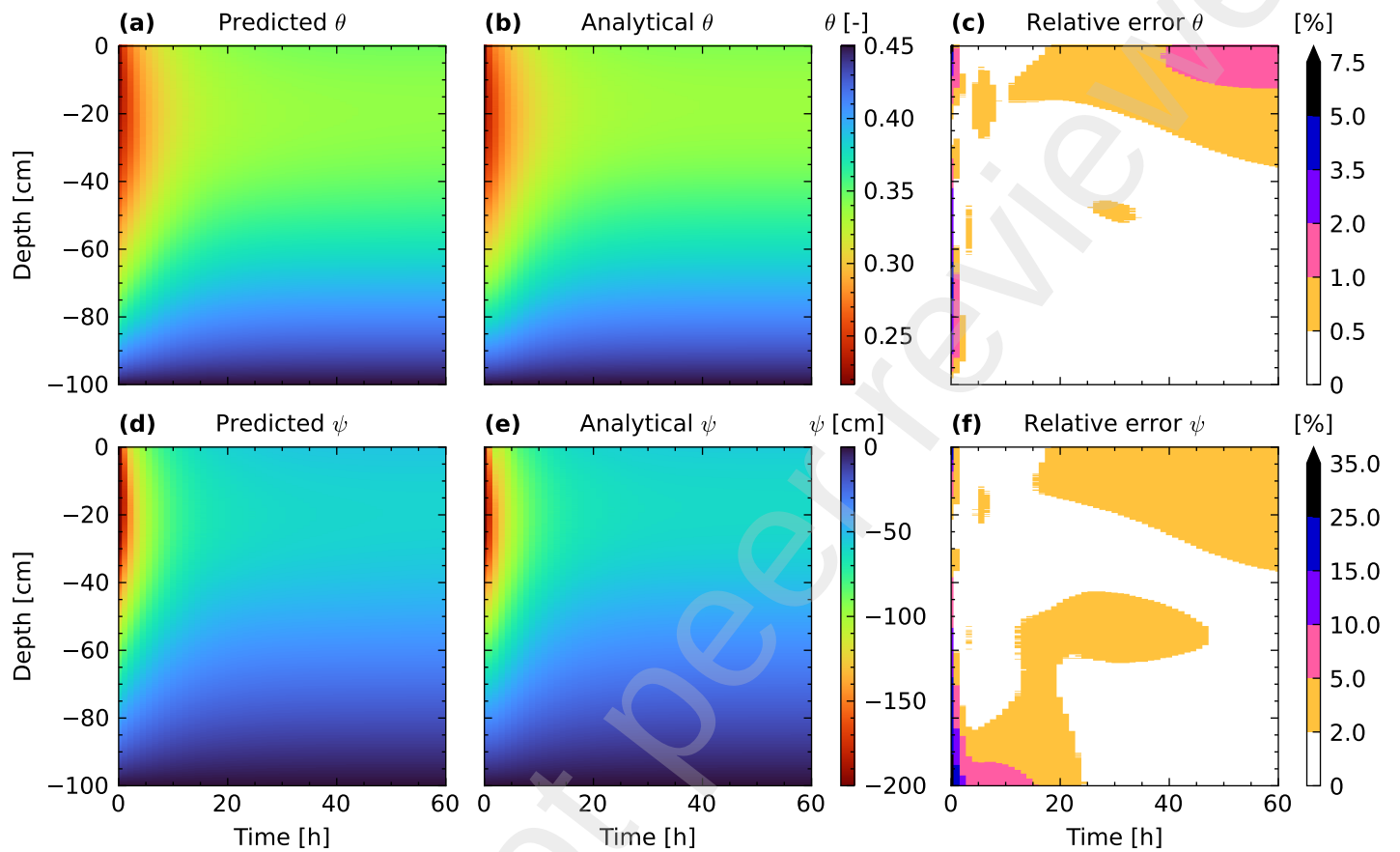


Figure 10: Spatio-temporal comparison of water content ( $\theta$ ) and pressure head ( $\psi$ ) for uniform root water uptake. PINN predictions (a, d), analytical solution (b, e), and relative error maps (c, f), respectively. The PINN model was trained with 5 sparse measurement points.

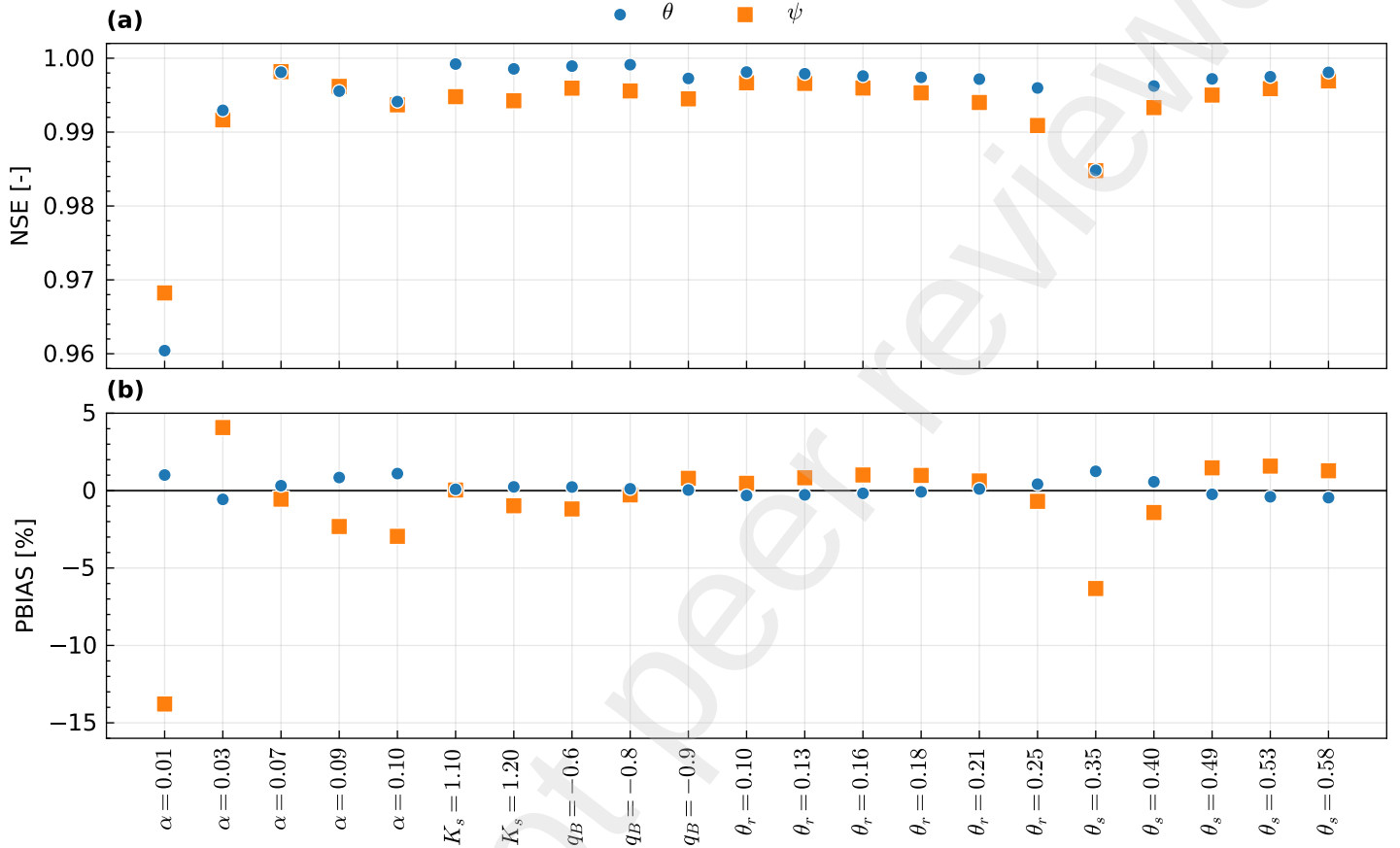


Figure 11: Sensitivity analysis of the P-PINNs for bare soil. The plots display the performance metrics for theta (blue circles) and  $\psi$  (orange squares) when evaluating different scenarios: (a) Nash-Sutcliffe Efficiency (NSE) and (b) Percent Bias (PBIAS). The parameters being changed from the reference scenario are stacked on the x-axis.

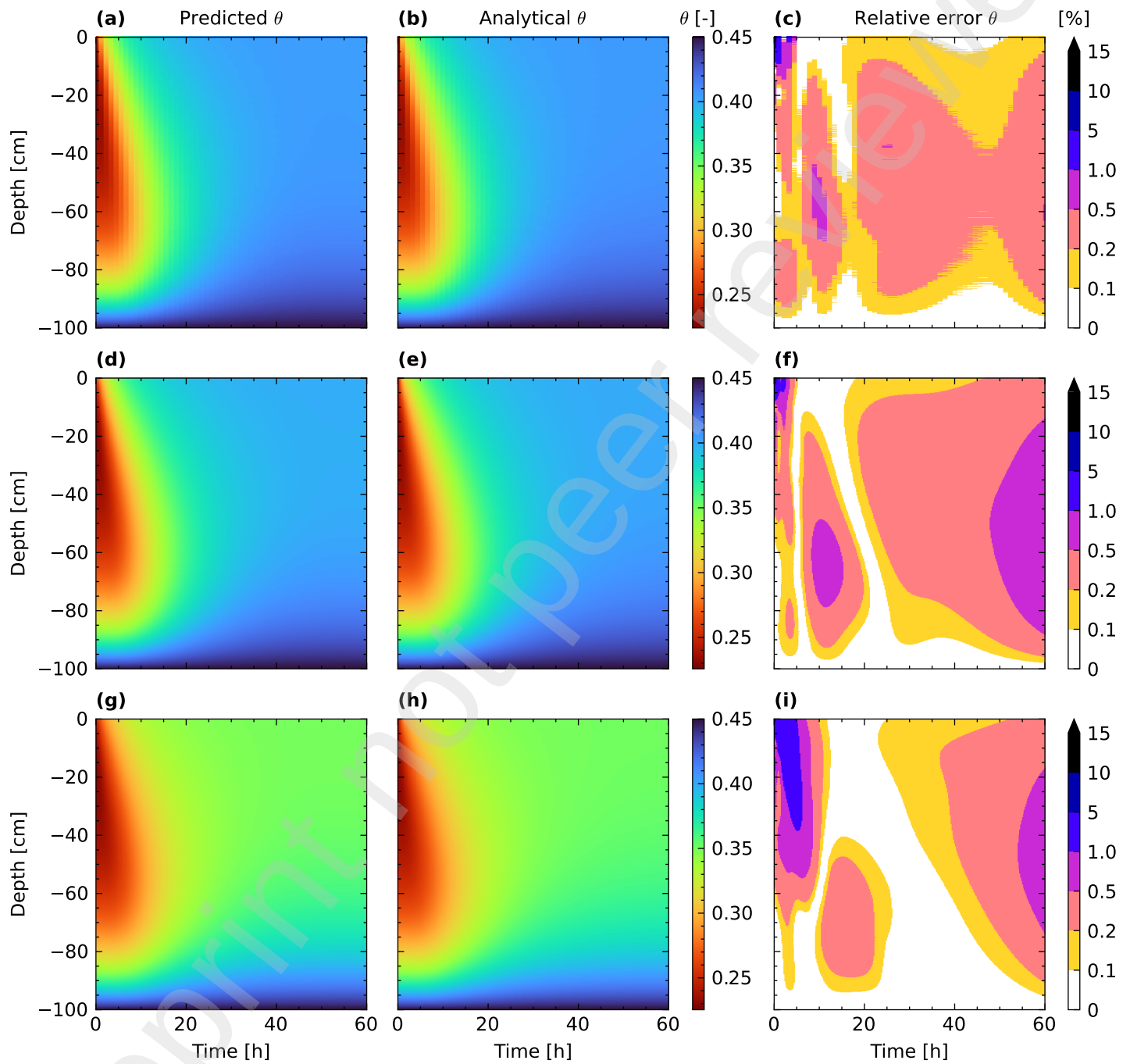


Figure 12: Spatio-temporal comparison of water content ( $\theta$ ) for the top-performing P-PINN scenarios in bare soil infiltration. PINN predictions, analytical solution, and relative error maps for  $k_s = 1.10 \text{ cm h}^{-1}$  (a, b, c),  $q_b = 0.8 \text{ cm h}^{-1}$  (d, e, f), and  $q_b = 0.8 \text{ cm h}^{-1}$  (g, h, i), respectively.

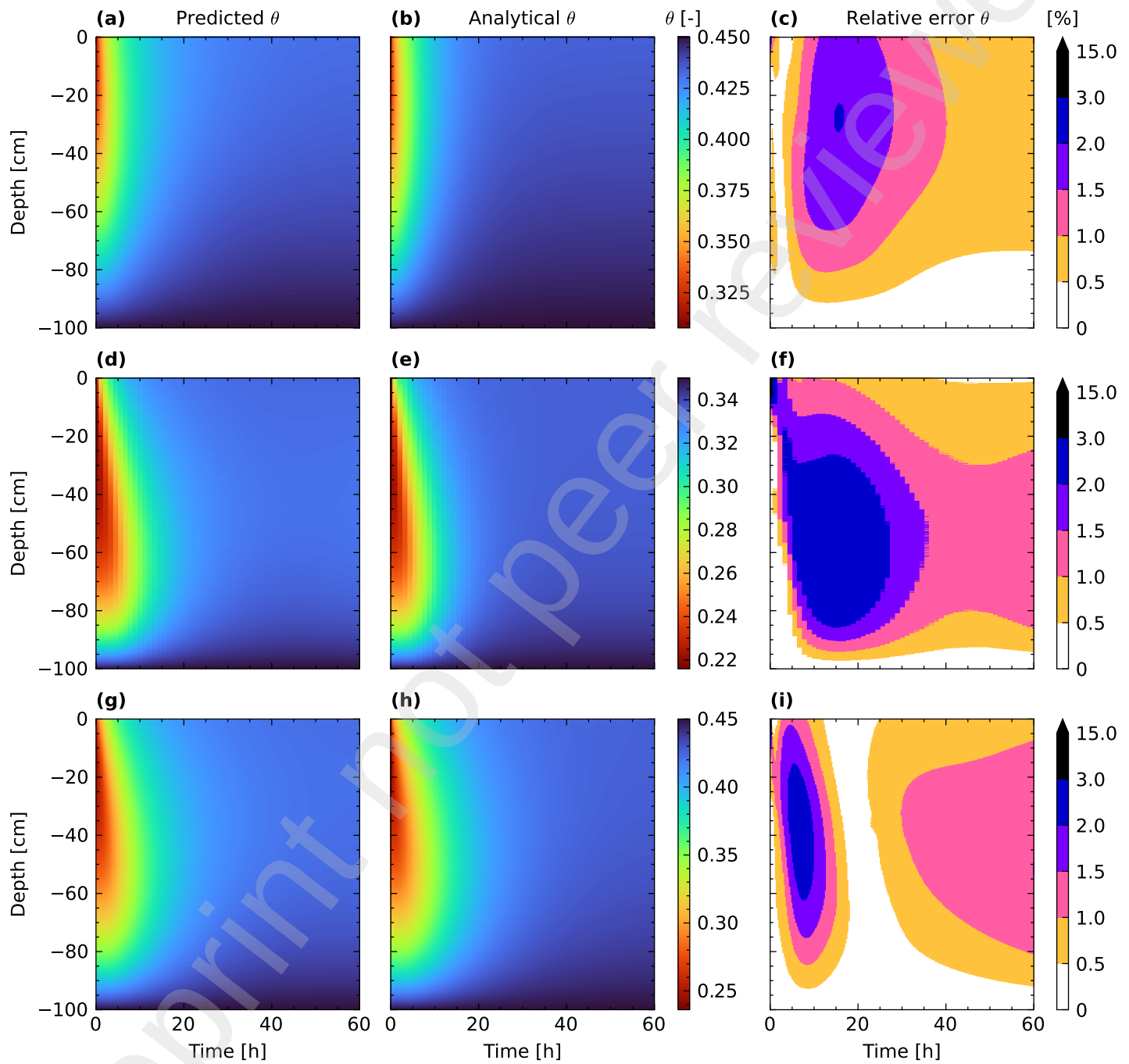


Figure 13: Spatio-temporal comparison of water content ( $\theta$ ) for the worst-performing P-PINN scenarios in bare soil infiltration. PINN predictions, analytical solution, and relative error maps for  $\alpha = 0.01 \text{ cm}^{-1}$  (a, b, c),  $\theta_s = 0.35$  (d, e, f), and  $\alpha = 0.03 \text{ cm}^{-1}$  (g, h, i), respectively.

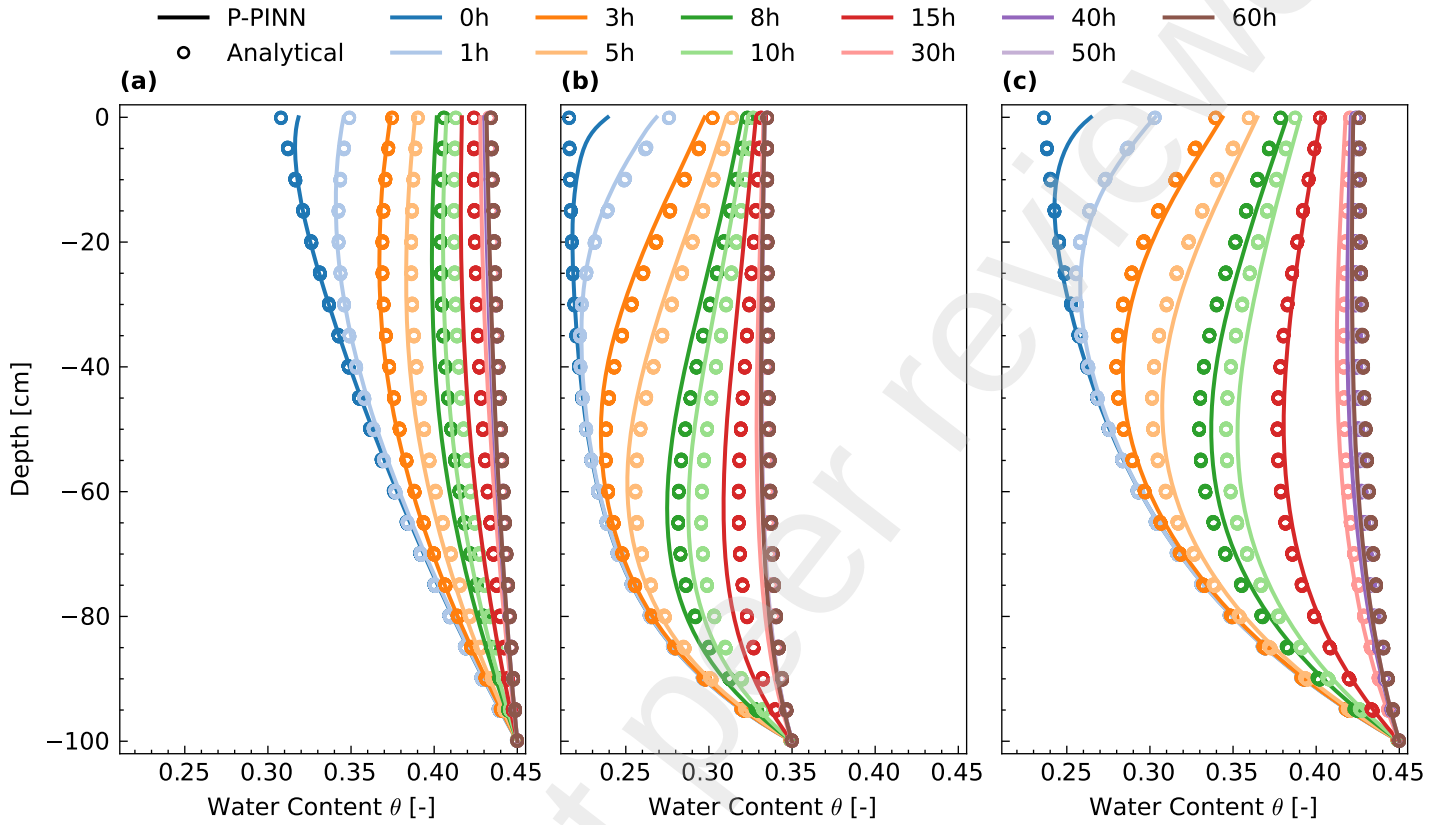


Figure 14: Vertical profiles of water content ( $\theta$ ) at selected time snapshots for bare soil infiltration for the three worst-performing scenarios  $\alpha = 0.01 \text{ cm}^{-1}$  (a),  $\theta_s = 0.35$  (b), and  $\alpha = 0.03 \text{ cm}^{-1}$  (c). Solid lines and open circles represent P-PINN predictions and analytical solutions, respectively.

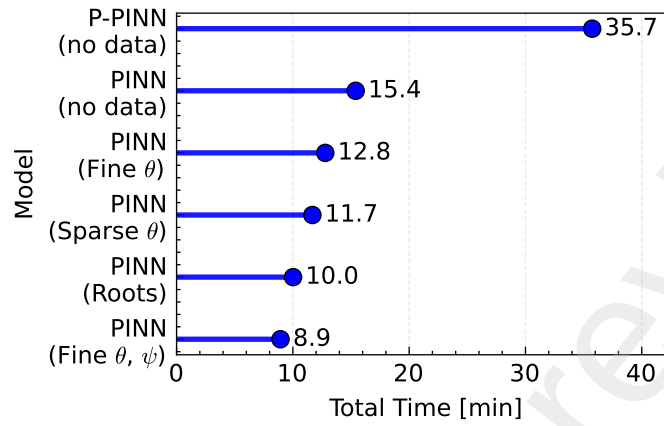


Figure 15: Comparison of the total computational cost between the standard PINN and P-PINN frameworks on 1 NVIDIA L40S GPU with 48 GB of vRAM, of which 44.4 GB are user-accessible, using CUDA 12.8, and PyTorch 2.7.1.

Table 1: Parameter ranges defining the training space  $\mathcal{P}$  for the P-PINNs.

| Parameter                    | Symbol     | Range        | Unit               |
|------------------------------|------------|--------------|--------------------|
| — Hydraulic Parameters —     |            |              |                    |
| Shape Parameter              | $\alpha$   | [0.01, 0.10] | $\text{cm}^{-1}$   |
| Saturated Conductivity       | $K_s$      | [1.0, 1.2]   | $\text{cm h}^{-1}$ |
| Residual Water Content       | $\theta_r$ | [0.10, 0.25] | —                  |
| Saturated Water Content      | $\theta_s$ | [0.35, 0.58] | —                  |
| — Upper Boundary Condition — |            |              |                    |
| Rainfall Flux                | $q_B$      | [-0.9, -0.6] | $\text{cm h}^{-1}$ |

Table 2: Baseline configuration of the Analytical Reference Dataset.

| Parameter                                           | Symbol         | Value                               |
|-----------------------------------------------------|----------------|-------------------------------------|
| — RRE & Gardner's Model —                           |                |                                     |
| Saturated Conductivity                              | $K_s$          | $1.0 \text{ cm h}^{-1}$             |
| Saturated Water Content                             | $\theta_s$     | $0.45 \text{ cm}^3 \text{ cm}^{-3}$ |
| Residual Water Content                              | $\theta_r$     | $0.20 \text{ cm}^3 \text{ cm}^{-3}$ |
| Shape Parameter                                     | $\alpha_{ref}$ | $0.01 \text{ cm}^{-1}$              |
| Sink Term (root uptake case only)                   | $S_0$          | $0.02 \text{ h}^{-1}$               |
| — Domain, Initial Condition & Boundary Conditions — |                |                                     |
| Spatial Domain                                      | $z$            | $[-100, 0] \text{ cm}$              |
| Temporal Domain                                     | $t$            | $[0, 60] \text{ h}$                 |
| Top Boundary at $t = 0$                             | $q_A$          | $-0.1 \text{ cm h}^{-1}$            |
| Top Boundary at $t > 0$                             | $q_B(t)$       | $-0.9 \text{ cm h}^{-1}$            |
| Bottom Boundary                                     | $\psi_{bot}$   | $0 \text{ cm}$                      |

Table 3: Hyperparameter configuration differences between PINN and P-PINN.

| Hyperparameter                             | Standard PINN               | P-PINN               |
|--------------------------------------------|-----------------------------|----------------------|
| — Scope & Network Architecture —           |                             |                      |
| Input Space                                | 2D                          | 7D                   |
| Network Size                               | $5L \times 128N$            | $6L \times 128N$     |
| — Sampling & Scenarios —                   |                             |                      |
| Training Scenarios                         | 1                           | 225                  |
| Collocation Points ( $N_f$ )               | 20,000                      | 50,000               |
| Top Boundary Points ( $N_{BC}$ )           | 3,000                       | 15,000               |
| Initial Condition Points ( $N_{IC}$ )      | 3,000                       | 20,000               |
| — Training Schedule —                      |                             |                      |
| Adam Iterations                            | 20,000                      | 22,000               |
| L-BFGS Iterations                          | 8,000                       | 3,000                |
| Initial Learning Rate                      | $1.0 \times 10^{-3}$        | $3.0 \times 10^{-4}$ |
| Minimum Learning Rate                      | $1.0 \times 10^{-6}$        | $1.0 \times 10^{-7}$ |
| — Loss Function Weights —                  |                             |                      |
| PDE Loss Weight ( $w_f$ )                  | 1.0                         | 10.0                 |
| Top Boundary Loss Weight ( $w_{BC}$ )      | 1.0                         | 8.0                  |
| Initial Condition Loss Weight ( $w_{IC}$ ) | 1.0                         | 6.0                  |
| — Adaptive Refinement (RAR) —              |                             |                      |
| RAR Strategy                               | Standard                    | 4 Layers             |
| RAR Iterations                             | 20                          | 4                    |
| New Points per Iteration                   | 300                         | 5,000                |
| — Root Uptake —                            |                             |                      |
| Sink Term                                  | $S_0 = 0.02 \text{ h}^{-1}$ | None                 |

Table 4: Quantitative comparison of PINN performance for bare soil under different training scenarios.

| Training Scenario       | Water Content $\theta$ [-] |        | Pressure Head $\psi$ [cm] |        |
|-------------------------|----------------------------|--------|---------------------------|--------|
|                         | RMSE                       | $R^2$  | RMSE                      | $R^2$  |
| Physics-Only            | $3.77 \times 10^{-4}$      | 0.9999 | 0.173                     | 0.9999 |
| Sparse $\theta$ (5 pts) | $2.33 \times 10^{-4}$      | 0.9999 | 0.160                     | 0.9999 |
| Fine $\theta$ Only      | $1.52 \times 10^{-4}$      | 0.9999 | 0.128                     | 0.9999 |
| Fine $\theta + \psi$    | $1.39 \times 10^{-4}$      | 0.9999 | 0.121                     | 0.9999 |

Table 5: Quantitative comparison of P-PINN performance.

|               | <b>Network Architecture (<math>L \times N</math>)</b> |                                 |                                  |
|---------------|-------------------------------------------------------|---------------------------------|----------------------------------|
| <b>Metric</b> | <b><math>4 \times 64</math></b>                       | <b><math>6 \times 64</math></b> | <b><math>6 \times 128</math></b> |
| MAE           | 0.012 053                                             | 0.012 038                       | 0.004 923                        |

*Appendix B.1. Supplementary Figures*

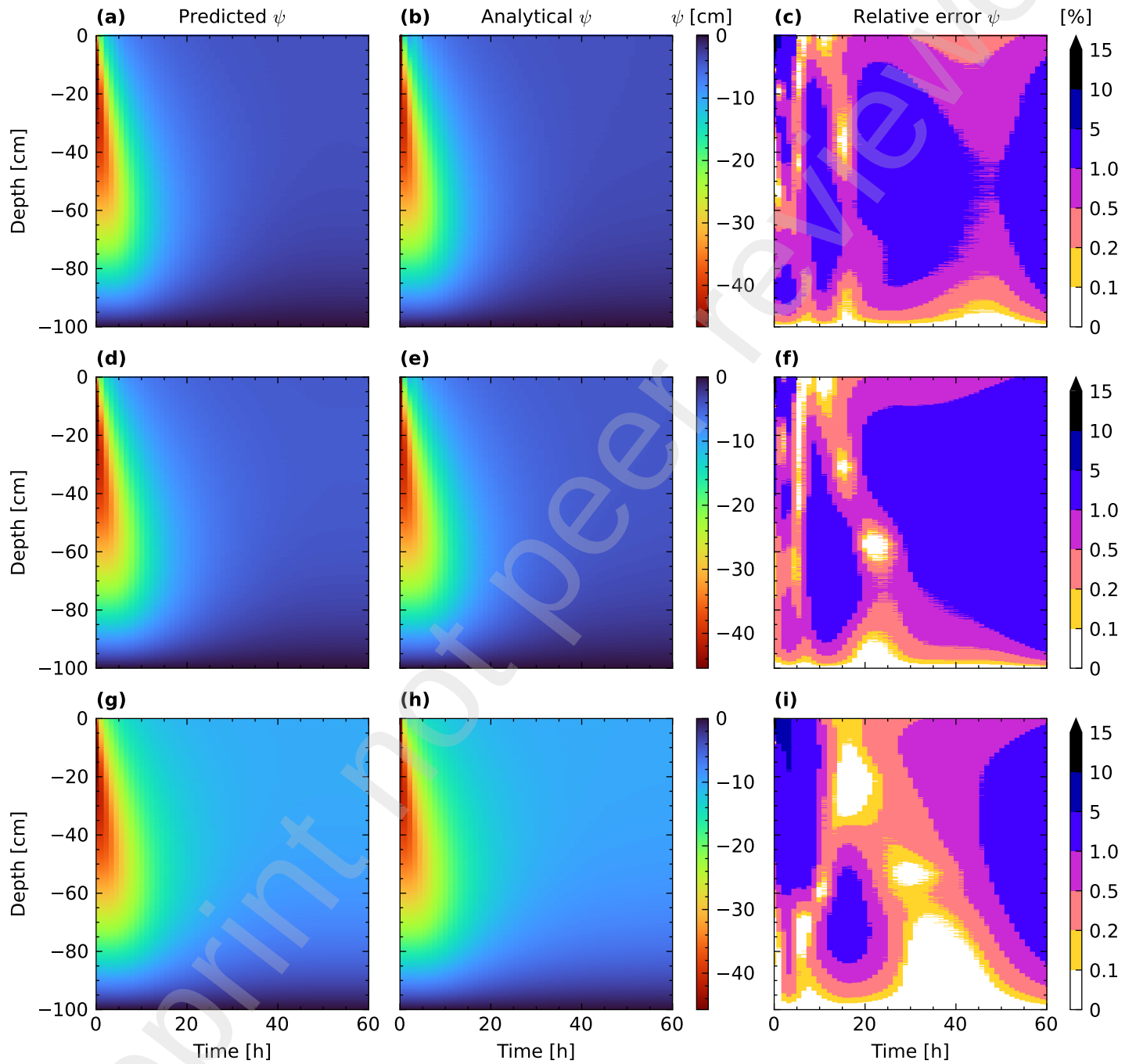


Figure 16: Spatio-temporal comparison of pressure head ( $\psi$ ) for the top-performing P-PINN scenarios in bare soil infiltration. PINN predictions, analytical solution, and relative error maps for  $k_s = 1.10 \text{ cm h}^{-1}$  (a, b, c),  $q_b = 0.8 \text{ cm h}^{-1}$  (d, e, f), and  $q_b = 0.8 \text{ cm h}^{-1}$  (g, h, i), respectively.

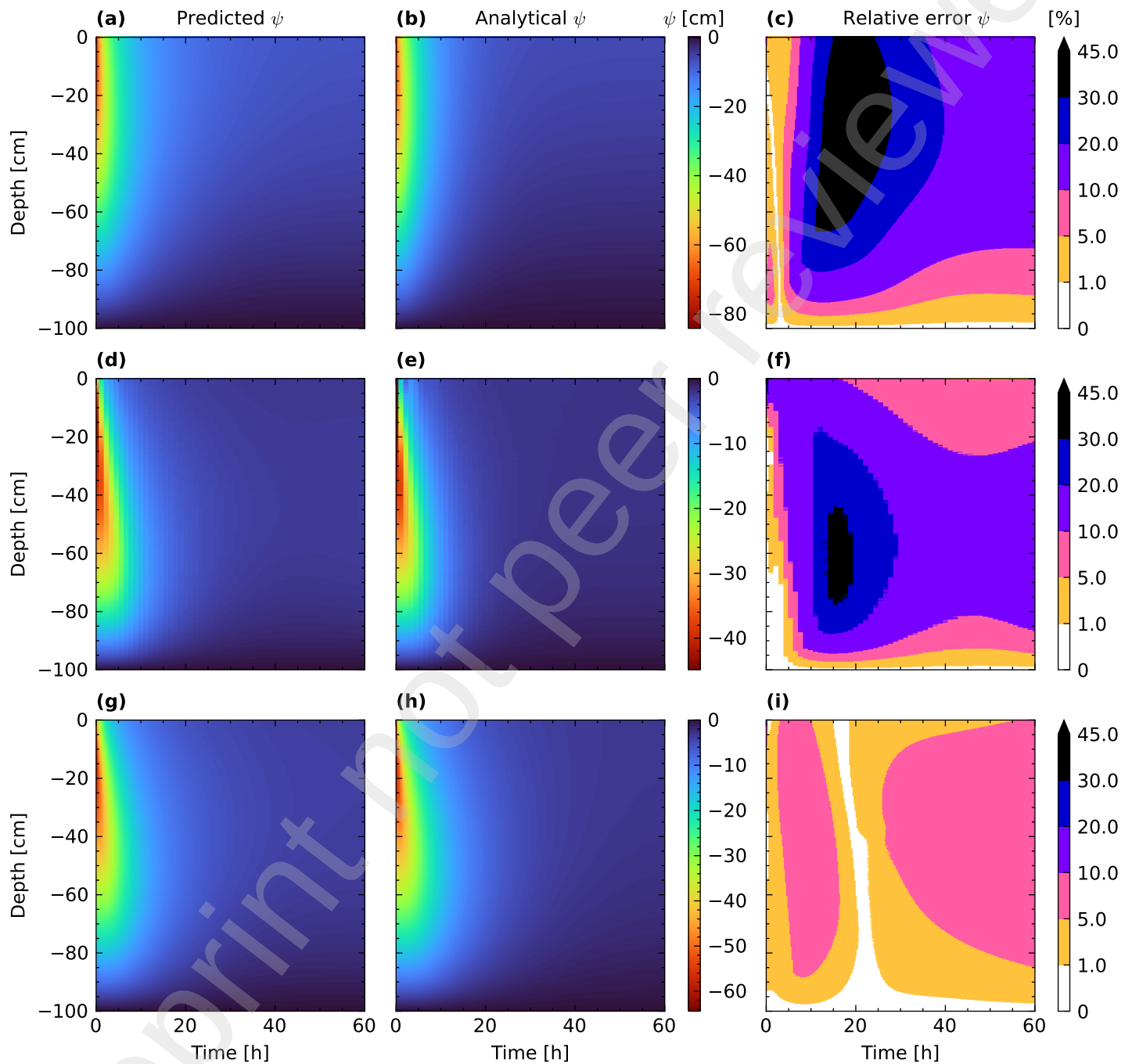


Figure 17: Spatio-temporal comparison of pressure head ( $\psi$ ) for the worst-performing P-PINN scenarios in bare soil infiltration. PINN predictions, analytical solution, and relative error maps for  $\alpha = 0.01 \text{ cm}^{-1}$  (a, b, c),  $\theta_s = 0.35$  (d, e, f), and  $\alpha = 0.03 \text{ cm}^{-1}$  (g, h, i), respectively.

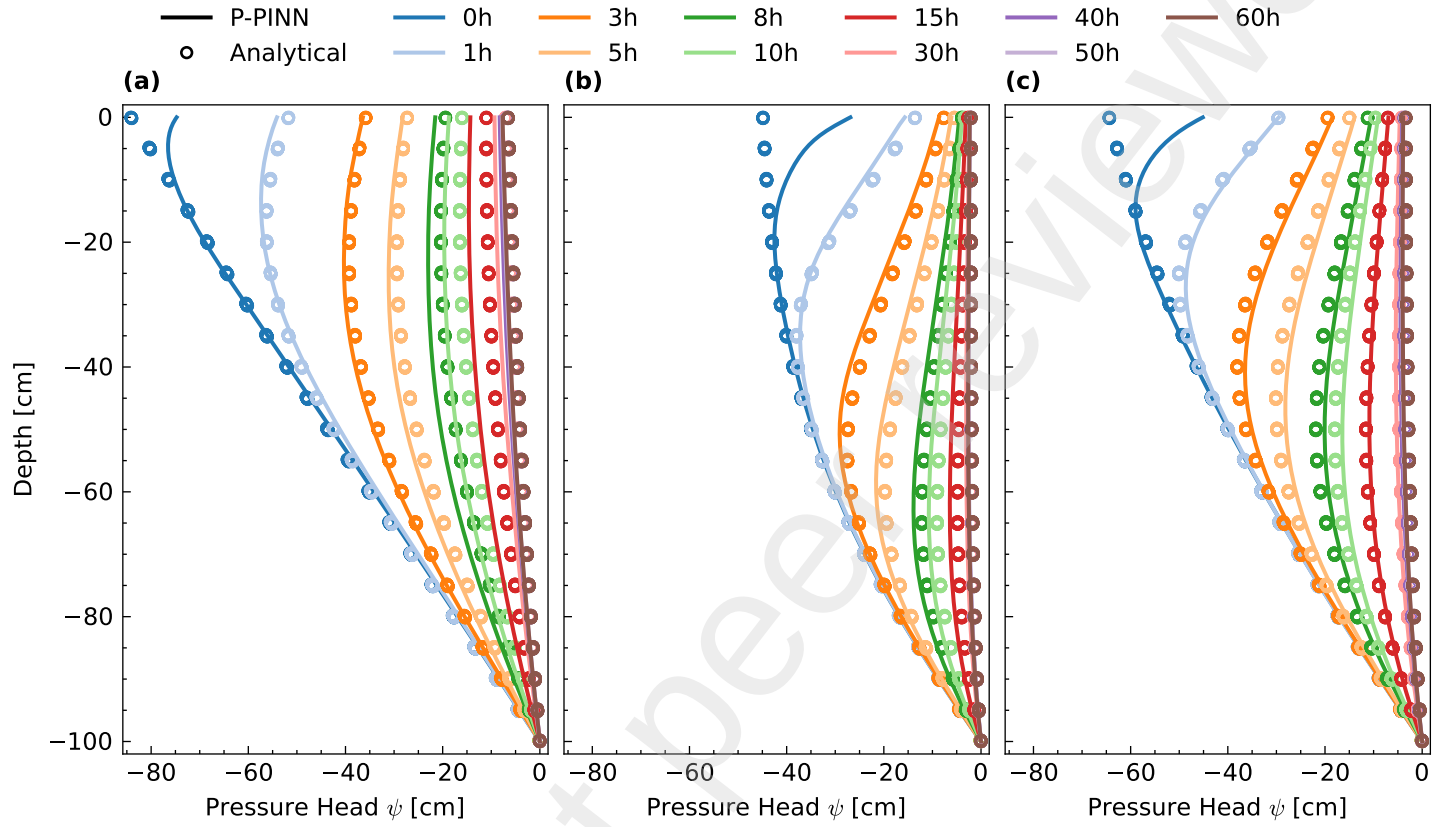


Figure 18: Vertical profiles of pressure head ( $\psi$ ) at selected time snapshots for bare soil infiltration for the three worst-performing scenarios  $\alpha = 0.01 \text{ cm}^{-1}$  (a),  $\theta_s = 0.35$  (b), and  $\alpha = 0.03 \text{ cm}^{-1}$  (c). Solid lines and open circles represent P-PINN predictions and analytical solutions, respectively.

## Appendix B.2. Supplementary Tables

Table 6: Summary of Evaluation Metrics used, adapted from [Moriasi et al. \(2007\)](#); [Raissi et al. \(2019\)](#).

| Metric                                 | Formula                                                                                                                                                                    |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mean Absolute Error (MAE)              | $\frac{1}{N} \sum_{i=1}^N  \hat{y}_i - y_i $                                                                                                                               |
| Root Mean Square Error (RMSE)          | $\sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}$                                                                                                                      |
| Pointwise Relative Error (%)           | $100 \cdot \frac{ \hat{y}_i - y_i }{\max( y_i , \epsilon)}$                                                                                                                |
| Coefficient of Determination ( $R^2$ ) | $\left( \frac{\sum_{i=1}^N (\hat{y}_i - \bar{\hat{y}})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (\hat{y}_i - \bar{\hat{y}})^2} \sqrt{\sum_{i=1}^N (y_i - \bar{y})^2}} \right)^2$ |
| Percent Bias (PBIAS)                   | $100 \cdot \frac{\sum_{i=1}^N (y_i - \hat{y}_i)}{\sum_{i=1}^N y_i}$                                                                                                        |
| Nash-Sutcliffe Efficiency (NSE)        | $1 - \frac{\sum_{i=1}^N (\hat{y}_i - y_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$                                                                                              |